



# SMART CAMPUS

Manual Técnico do Sistema

---

DOCUMENTAÇÃO INTERNA · EQUIPE DE DESENVOLVIMENTO



## Ficha Técnica do Documento

| Campo                         | Descrição                                                                                                |
|-------------------------------|----------------------------------------------------------------------------------------------------------|
| Título                        | Manual Técnico do Sistema Smart Campus                                                                   |
| Escopo                        | Arquitetura, linguagem SCEDS, módulos funcionais, instalação, operação, manutenção e evolução do sistema |
| Público-alvo                  | Equipes de desenvolvimento, manutenção e auditoria técnica responsáveis pela continuidade do sistema     |
| Versão do sistema documentada | 1.0.0                                                                                                    |
| Plataforma de referência      | Servidor Windows dedicado, executado na rede local da instituição                                        |
| Linguagem de implementação    | Python 3 (Flask), com camada de persistência própria denominada SCEDS                                    |
| Natureza do documento         | Referência técnica permanente, sujeita a revisão e controle de versão                                    |
| Classificação                 | Uso interno — não destinado à distribuição externa                                                       |

## Convenções Utilizadas neste Manual

Ao longo deste manual, blocos de código são apresentados em fonte monoespaçada sobre fundo escuro. Comandos da linguagem SCEDS são grafados em letras maiúsculas quando representam palavras reservadas. Nomes de arquivos, diretórios, variáveis de configuração e endpoints HTTP são apresentados em fonte monoespaçada inline, como em `core/config.json`.

Três tipos de quadros explicativos são utilizados: quadros de **nota**, que trazem esclarecimentos complementares; quadros de **aviso**, que sinalizam riscos operacionais ou pontos de atenção obrigatória; e quadros de **exemplo**, que apresentam casos de uso comentados.

## Sumário de Módulos do Sistema

| Módulo              | Prefixo de Rota                                      | Função                                                            |
|---------------------|------------------------------------------------------|-------------------------------------------------------------------|
| Núcleo (core)       | —                                                    | Autenticação, sessão, configuração global e roteamento de módulos |
| SCEDS               | —                                                    | Camada de persistência de dados própria do sistema                |
| Administração       | <code>/admin</code>                                  | Gestão de usuários, backups e configurações                       |
| Agendamento         | <code>/agendamento</code>                            | Reserva de recursos físicos compartilhados                        |
| Biblioteca          | <code>/biblioteca</code>                             | Catálogo de acervo e controle de empréstimos                      |
| Ocorrências         | <code>/ocorrencias</code>                            | Registro disciplinar e de elogios de alunos                       |
| Secretaria/Portaria | <code>/secretaria</code> ,<br><code>/portaria</code> | Agenda de compromissos e comunicação interna por chat             |
| IoT                 | <code>/iot</code>                                    | Integração com sensores e atuadores físicos (ESP32)               |
| Sinal               | <code>/sinal</code>                                  | Controle automatizado do sinal sonoro escolar                     |

|               |                |                                                    |
|---------------|----------------|----------------------------------------------------|
| Monitoramento | /monitoramento | Painéis analíticos consolidados dos demais módulos |
|---------------|----------------|----------------------------------------------------|

# Apresentação do Sistema

O Smart Campus é um sistema de gestão integrada desenvolvido para atender às necessidades operacionais de uma instituição de ensino, unificando em uma única plataforma web os processos de controle de acesso, biblioteca, agendamento de recursos físicos, registro de ocorrências disciplinares, comunicação interna, automação de dispositivos físicos e emissão de sinais sonoros de troca de aula.

O sistema foi concebido para operar em ambiente de rede local, dispensando conexão obrigatória com a internet para seu funcionamento essencial. Um único servidor, instalado em um computador com sistema operacional Windows mantido ligado durante o período de funcionamento da instituição, concentra a aplicação web e a camada de persistência de dados. As estações de trabalho, tablets e smartphones utilizados pelas equipes da secretaria, portaria, biblioteca, coordenação e corpo docente acessam o sistema por meio de navegador web comum, sem necessidade de instalação de software adicional.

A plataforma é estruturada em torno de um núcleo de autenticação e roteamento e de um conjunto de módulos funcionais independentes, cada um responsável por um domínio específico da operação escolar. Essa organização modular permite que novos módulos sejam adicionados, ou módulos existentes sejam removidos ou substituídos, com impacto mínimo sobre o restante do sistema.

**NOTA**

Este manual documenta a versão 1.0.0 do sistema, tal como distribuída no pacote de instalação de referência. Alterações estruturais realizadas em versões futuras devem ser refletidas em revisões subsequentes deste documento.

## 1.1. Escopo Funcional

O sistema abrange, na versão documentada, os seguintes domínios funcionais: controle de acesso e perfis de usuário; reserva de recursos compartilhados (laboratórios, equipamentos, salas e similares); catálogo e empréstimo de acervo bibliográfico; registro estruturado de ocorrências disciplinares e elogios; agenda de compromissos e canal de comunicação entre secretaria e portaria; leitura e atuação sobre dispositivos físicos conectados via microcontroladores ESP32 (sensores de nível de água, portões motorizados e controladores de ar-condicionado); acionamento programado e manual do sinal sonoro de troca de aula; e consolidação analítica dos dados produzidos pelos demais módulos em painéis de monitoramento.

## 1.2. Destinatários deste Manual

Este documento destina-se exclusivamente a profissionais técnicos responsáveis pela manutenção corretiva, evolutiva e preventiva do sistema, bem como por auditorias de código, segurança e conformidade. Não constitui manual de utilização para usuários finais dos módulos (professores, portaria, biblioteca, secretaria e coordenação), os quais dispõem de interfaces autoexplicativas e não requerem conhecimento técnico para operação cotidiana.

# Objetivos do Projeto

---

O desenvolvimento do Smart Campus foi orientado pelos objetivos descritos nesta seção, os quais devem ser preservados em qualquer trabalho de evolução futura do sistema.

## 2.1. Objetivo Geral

Substituir processos manuais, dispersos em cadernos, planilhas isoladas e comunicação informal, por uma plataforma única, coerente e auditável, capaz de registrar de forma estruturada as operações do dia a dia escolar e de integrar-se a dispositivos físicos de automação.

## 2.2. Objetivos Específicos

- Centralizar a autenticação de usuários por perfil funcional, evitando duplicidade de credenciais entre módulos.
- Eliminar a dependência de serviços externos pagos ou de conexão permanente com a internet para as operações essenciais.
- Fornecer uma camada de persistência de dados própria, de auditoria simples (arquivos legíveis) e sem dependências de bancos de dados externos que exijam instalação, licenciamento ou administração especializada.
- Padronizar a criação de novos módulos por meio de convenções de estrutura, nomenclatura e integração com o núcleo do sistema.
- Permitir a operação de dispositivos IoT de baixo custo (ESP32) para automação de tarefas físicas — abertura de portões, monitoramento de caixas de água e controle de aparelhos de ar-condicionado — sem exigir infraestrutura de nuvem.
- Registrar de forma auditável o histórico disciplinar dos alunos e o uso dos recursos compartilhados da instituição.

## 2.3. Fora do Escopo

Não constituem objetivo do sistema, na versão documentada: integração com sistemas de folha de pagamento ou financeiros; emissão de documentos acadêmicos oficiais (históricos, certificados); e operação em múltiplas unidades escolares a partir de uma única instalação — cada instalação do Smart Campus atende a uma única instituição.

# Arquitetura Geral

O Smart Campus adota uma arquitetura monolítica modular: uma única aplicação Flask concentra o roteamento HTTP, mas o código é segmentado em módulos independentes conectados ao núcleo por meio de um mecanismo de registro de blueprints. Essa escolha arquitetural privilegia a simplicidade de instalação e operação — um único processo Python a ser mantido em execução — em detrimento da escalabilidade horizontal, que não é requisito do sistema dado o contexto de uso (uma única instituição, rede local, carga concorrente reduzida).

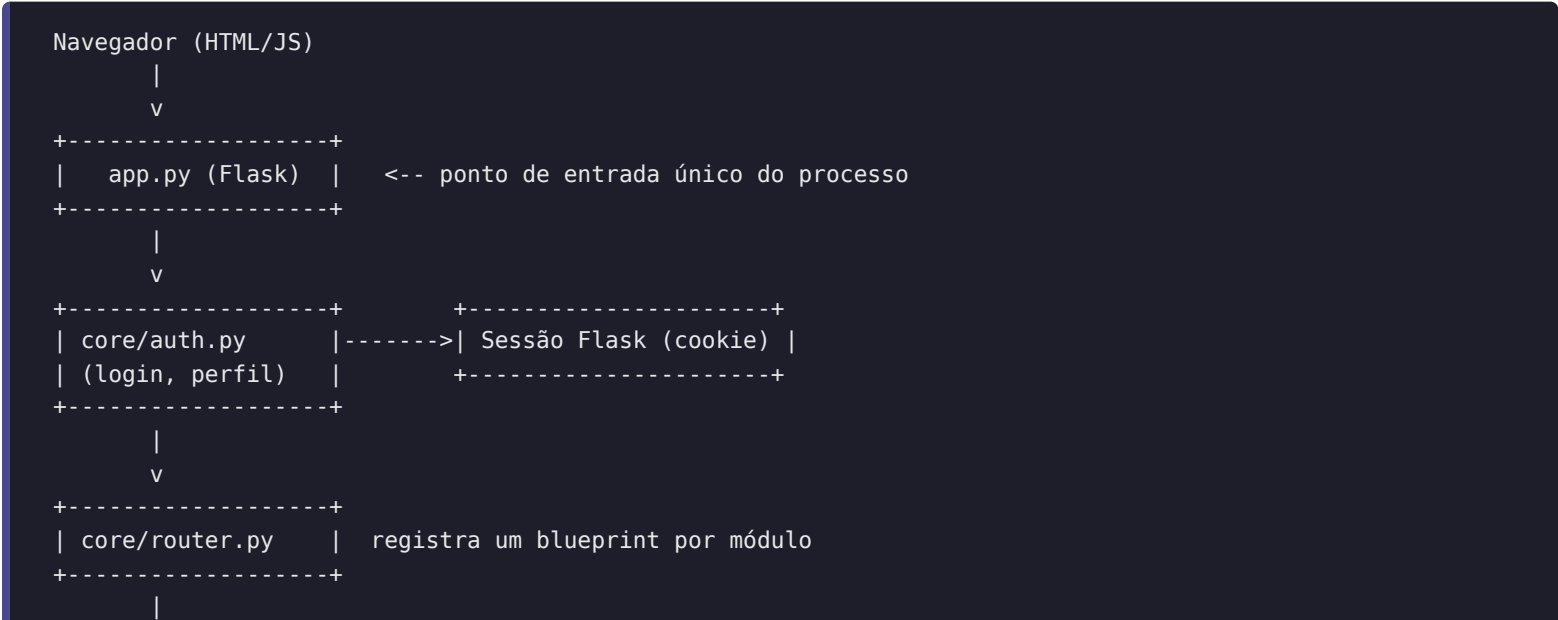
## 3.1. Camadas do Sistema

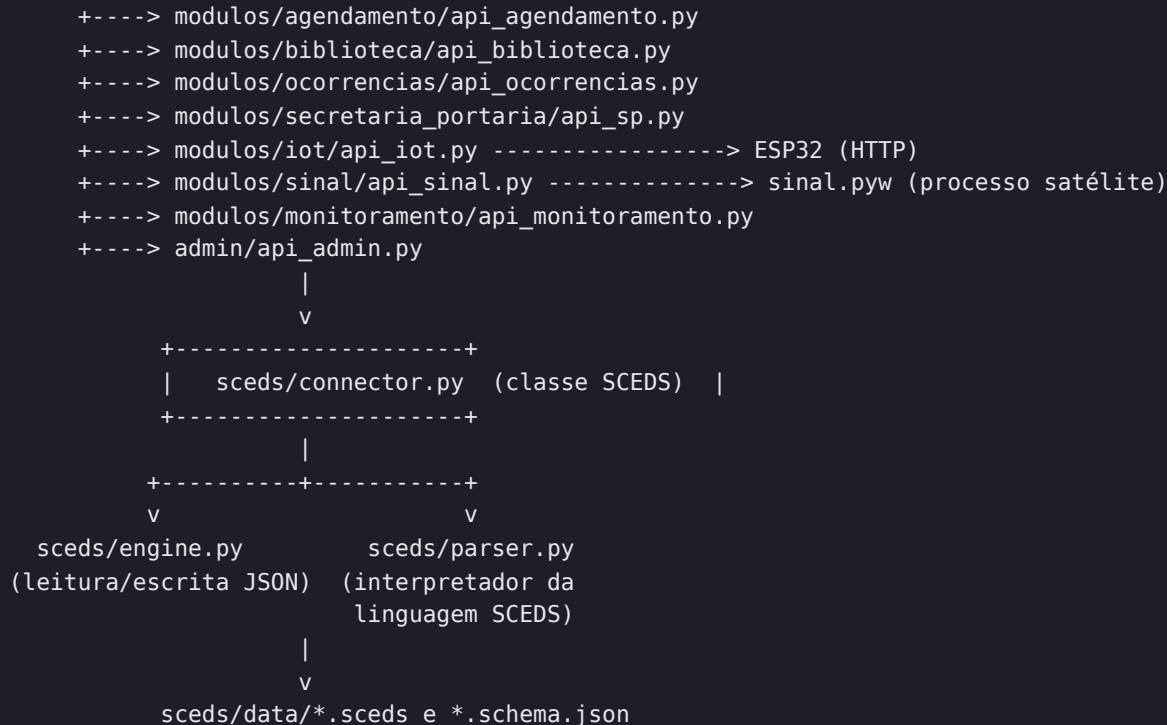
| Camada               | Responsabilidade                                                                                 | Localização no código                                           |
|----------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Apresentação         | Páginas HTML renderizadas por Jinja2 e ativos estáticos (CSS/JS)                                 | templates/, static/                                             |
| Roteamento e API     | Endpoints HTTP de cada módulo, expostos como blueprints Flask                                    | modulos/*/api_*.py, admin/api_admin.py                          |
| Núcleo (core)        | Autenticação, sessão, controle de acesso por perfil, registro de blueprints, configuração global | core/                                                           |
| Persistência         | Motor de armazenamento e linguagem de consulta SCEDS                                             | sceds/                                                          |
| Dados                | Arquivos de tabela e schema em formato JSON                                                      | sceds/data/                                                     |
| Dispositivos físicos | Firmware dos microcontroladores ESP32 e simulador de desenvolvimento                             | modulos/iot/firmware/,<br>modulos/iot/simulador_iot_completo.py |

## 3.2. Diagrama de Componentes

### NOTA DE LEITURA

O diagrama a seguir representa o fluxo de uma requisição típica, desde o navegador do usuário até a camada de dados, passando pelo núcleo de autenticação e pelo módulo funcional correspondente.





### 3.3. Princípios Arquiteturais

- **Um processo, um servidor.** Toda a aplicação web roda em um único processo Flask, iniciado por `python app.py`. Não há orquestração de múltiplos serviços, exceto o processo satélite do módulo Sinal (ver Capítulo 10).
- **Módulos desacoplados por contrato de blueprint.** Cada módulo expõe uma variável `blueprint` em seu arquivo de API; o núcleo não conhece detalhes internos de nenhum módulo, apenas importa esse contrato.
- **Falha isolada de módulo não interrompe o servidor.** O registrador de blueprints captura exceções de importação por módulo e prossegue com os demais, registrando o erro em log.
- **Persistência sem servidor de banco de dados externo.** A camada SCEDS lê e escreve arquivos JSON diretamente no disco do servidor, eliminando a necessidade de instalar, licenciar ou administrar um sistema gerenciador de banco de dados convencional.



# Estrutura de Diretórios

A distribuição do sistema segue a organização de pastas descrita a seguir. A raiz do pacote de instalação contém o instalador e a pasta do produto, mantidos lado a lado.

```
(pasta de instalação)/
├── instalar.py                Instalador interativo (linha de comando)
├── SmartCampus/
│   ├── app.py                Ponto de entrada da aplicação Flask
│   ├── core/
│   │   ├── auth.py           Autenticação, sessão e controle de perfil
│   │   ├── router.py         Registro de blueprints dos módulos
│   │   ├── config.json       Configuração global da instalação
│   │   └── .secret_key        Chave de assinatura de sessão (gerada em tempo de execução)
│   ├── sceds/
│   │   ├── engine.py         Motor de leitura/escrita das tabelas
│   │   ├── parser.py         Interpretador da linguagem SCEDS
│   │   ├── connector.py      Classe de fachada (SCEDS)
│   │   └── data/             Arquivos .sceds e .schema.json de cada tabela
│   ├── modulos/
│   │   ├── agendamento/
│   │   ├── biblioteca/
│   │   ├── ocorrencias/
│   │   ├── secretaria_portaria/
│   │   ├── iot/
│   │   │   ├── firmware/     Código-fonte .ino para os ESP32
│   │   │   └── hardware/     Planejamento de hardware e ligações
│   │   ├── sinal/
│   │   │   └── sons/          Arquivo de áudio do sinal sonoro
│   │   └── monitoramento/
│   ├── admin/
│   │   └── api_admin.py       Módulo administrativo (usuários, backup)
│   ├── templates/            Páginas HTML (Jinja2), uma subpasta por módulo
│   ├── static/
│   │   ├── css/style.css
│   │   └── js/main.js
│   └── logs/                  Arquivos de log gerados em tempo de execução
```

## 4.1. Convenção de Módulo

Todo módulo funcional reside em uma subpasta de `modulos/` e segue, no mínimo, a seguinte convenção interna:

| Arquivo/Pasta                               | Finalidade                                                                                            |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>                    | Torna a pasta um pacote Python importável                                                             |
| <code>api_&lt;módulo&gt;.py</code>          | Define o <code>blueprint</code> Flask e as rotas do módulo                                            |
| arquivos <code>.json</code> de configuração | Dados estruturais próprios do módulo que não pertencem ao SCEDS (ex.: turmas, recursos, dispositivos) |

NOTA

Os arquivos de dados do SCEDS residem exclusivamente em `sceds/data/`, centralizados independentemente do módulo que os utiliza. Arquivos de configuração auxiliares específicos de um módulo (como `turmas.json` do módulo Ocorrências ou `recursos.json` do módulo Agendamento) residem na própria pasta do módulo.

# Fluxo de Funcionamento

## 5.1. Ciclo de Vida de uma Requisição

Toda requisição HTTP recebida pelo processo Flask segue o mesmo percurso geral, independentemente do módulo de destino:

- 1. O servidor WSGI interno do Flask recebe a requisição na porta configurada em `core/config.json` (campo `porta_api`).
- 2. O contexto de aplicação injeta, via `injetar_contexto()` em `app.py`, os dados de identidade da escola e do usuário autenticado (se houver), disponíveis a todos os templates renderizados.
- 3. Rotas protegidas verificam a sessão por meio dos decoradores `login_obrigatorio` ou `perfil_obrigatorio(*perfis)`, definidos em `core/auth.py`.
- 4. A rota do módulo correspondente executa sua lógica, tipicamente invocando a camada SCEDS para leitura ou escrita de dados.
- 5. A resposta é serializada como HTML (páginas renderizadas por Jinja2) ou como JSON (endpoints prefixados por `/api/`, consumidos pelo JavaScript de cada painel).

## 5.2. Fluxo de Autenticação

O sistema utiliza autenticação por senha única, sem campo de usuário visível no formulário de login: o servidor identifica o usuário testando a senha informada contra o hash armazenado de todos os usuários ativos, retornando o primeiro que corresponder. Esse comportamento está implementado em `autenticar()`, em `core/auth.py`.

**AVISO**

Por decorrência direta desse desenho, o sistema impõe a regra de que nenhuma senha pode coincidir com a de outro usuário — caso contrário, a identificação do usuário correto na etapa de login se tornaria ambígua. Essa verificação de unicidade é aplicada em `criar_usuario()` e em `redefinir_senha()`. Qualquer alteração futura que reintroduza um campo de nome de usuário deve reavaliar essa restrição.

## 5.3. Fluxo Típico de um Módulo Funcional

- 1. Usuário autenticado acessa a URL do módulo (ex.: `/biblioteca/`)
- 2. Blueprint do módulo verifica o perfil via `@perfil_obrigatorio(...)`
- 3. Template HTML é renderizado com o "esqueleto" da página
- 4. JavaScript da página consulta endpoints `/api/...` do próprio módulo
- 5. Cada endpoint invoca métodos da classe SCEDS (`buscar/inserir/atualizar/deletar`)
- 6. SCEDS lê ou grava os arquivos `.sceds` correspondentes em `sceds/data/`
- 7. Resposta JSON retorna ao navegador, que atualiza a interface dinamicamente

## 5.4. Fluxo dos Dispositivos IoT

Diferentemente dos demais módulos, o módulo IoT recebe também requisições originadas de dispositivos físicos (ESP32), e não apenas de navegadores. Dois padrões de comunicação coexistem:

- **Push do dispositivo para o servidor:** sensores de nível de água enviam leituras periódicas via `POST /iot/agua/<sensor_id>`, autenticadas por token no cabeçalho `X-Device-Token`.
- **Polling do dispositivo ao servidor:** portões e controladores de ar-condicionado consultam periodicamente um endpoint de comando (ex.: `GET /iot/portao/<pid>/comando`) para saber se há uma ação pendente, e reportam de volta o estado físico real após executá-la.

Um agendador de segundo plano, implementado como uma thread daemon iniciada junto ao processo Flask (`_loop_automacoes()`), verifica a cada vinte segundos se algum horário programado de abertura de portão, fechamento de portão ou acionamento de ar-condicionado foi atingido, atualizando o estado desejado consultado pelo polling dos dispositivos.

# Tecnologias Utilizadas

| Camada                               | Tecnologia                                            | Finalidade                                                                                        |
|--------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Linguagem principal                  | Python 3                                              | Implementação de todo o backend, do motor SCEDS e dos scripts de instalação/manutenção            |
| Framework web                        | Flask                                                 | Roteamento HTTP, sessões, renderização de templates Jinja2                                        |
| Motor de templates                   | Jinja2 (embutido no Flask)                            | Composição das páginas HTML a partir de templates/                                                |
| Hashing de senha                     | bcrypt, com fallback para SHA-256 com salt            | Armazenamento seguro de credenciais quando o pacote nativo bcrypt não está disponível no ambiente |
| Persistência                         | SCEDS (implementação própria sobre arquivos JSON)     | Armazenamento estruturado de todas as entidades do sistema                                        |
| Front-end                            | HTML5, CSS3, JavaScript (ES6, sem framework de build) | Interfaces dos painéis, consumindo os endpoints JSON de cada módulo                               |
| Comunicação com dispositivos         | HTTP simples sobre a rede local                       | Leitura de sensores e envio de comandos aos ESP32                                                 |
| Firmware dos dispositivos            | C++ (Arduino/ESP32, arquivos .ino)                    | Lógica embarcada dos sensores de água, portões e controlador de ar-condicionado                   |
| Notificação externa                  | pywhatkit                                             | Envio de mensagens de WhatsApp pelo módulo Sinal (saída antecipada)                               |
| Integração com o sistema operacional | pywin32                                               | Registro do servidor como inicialização automática no Windows                                     |
| Empacotamento de backups             | zipfile (biblioteca padrão do Python)                 | Compactação dos backups completos gerados pelo módulo Administração                               |

## NOTA

O sistema foi deliberadamente projetado sem dependência de bancos de dados relacionais (PostgreSQL, MySQL, SQLite) ou de serviços de nuvem. Essa decisão reduz a superfície de manutenção em ambientes escolares que, tipicamente, não dispõem de administrador de banco de dados dedicado, mas implica as limitações de escala e concorrência descritas no Capítulo 20.

# Padrões de Desenvolvimento

## 7.1. Padrão de Módulo (Blueprint Contract)

Todo módulo novo deve expor, em seu arquivo principal de API, uma variável de nível de módulo chamada `blueprint`, instância de `flask.Blueprint`. O núcleo (`core/router.py`) importa esse nome por convenção; módulos que não o expõem são ignorados silenciosamente (com registro de aviso em log), e não interrompem a inicialização dos demais módulos.

Modelo de arquivo `api_<módulo>.py`

```
from flask import Blueprint, render_template, jsonify, request
from core.auth import login_obrigatorio, perfil_obrigatorio

blueprint = Blueprint("nome_do_modulo", __name__,
                      template_folder="../../templates/nome_do_modulo")

def _get_db():
    import sys, json
    from pathlib import Path
    sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))
    from sceds import SCEDS
    cfg = json.load(open(Path(__file__).resolve().parent.parent.parent
                             / "core" / "config.json", encoding="utf-8"))
    return SCEDS(Path(cfg["caminho_base"]) / "sceds" / "data")

@blueprint.route("/")
@login_obrigatorio
def index():
    return render_template("nome_do_modulo/index.html")
```

## 7.2. Padrão de Acesso a Dados

Módulos não devem manipular arquivos `.sceds` diretamente; toda leitura e escrita deve passar pela classe `SCEDS`, obtida por meio de uma função auxiliar local `_get_db()`, repetida (com pequenas variações) em cada módulo. Essa repetição é intencional na versão atual do sistema, para manter cada módulo independente de importações cruzadas; ver recomendação de consolidação no Capítulo 34.

## 7.3. Padrão de Controle de Acesso por Rota

Toda rota que exponha dados ou ações deve ser protegida por um dos dois decoradores de `core/auth.py`:

| Decorador                                                      | Uso recomendado                                                              |
|----------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>@login_obrigatorio</code>                                | Rotas acessíveis a qualquer usuário autenticado, independentemente do perfil |
| <code>@perfil_obrigatorio('admin', 'coordenadora', ...)</code> | Rotas restritas a um ou mais perfis específicos                              |

Ambos os decoradores retornam `401` em formato JSON quando a requisição é dirigida a um endpoint de API (caminho iniciado por `/api/` ou corpo declarado como JSON), e redirecionam para a tela de login nos demais

casos — preservando a experiência adequada tanto para chamadas de front-end quanto para navegação direta.

## 7.4. Padrão de Resposta de Erro

Endpoints de API devem retornar objetos JSON com a chave `"erro"` contendo mensagem legível em português, acompanhados do código de status HTTP apropriado (`401` para ausência de autenticação, `403` para perfil sem permissão, `404` para recurso inexistente, `500` para falhas internas).

# Convenções de Código

## 8.1. Idioma

Todo o código-fonte do sistema — nomes de funções, variáveis, classes, mensagens de log e de erro, comentários e strings de interface — é escrito em português. Essa convenção deve ser mantida em qualquer extensão futura, para preservar a coerência do projeto e a legibilidade para as equipes que o mantêm.

## 8.2. Nomenclatura

| Elemento                     | Convenção                                  | Exemplo                                                      |
|------------------------------|--------------------------------------------|--------------------------------------------------------------|
| Funções e variáveis          | <code>snake_case</code> em português       | <code>buscar_um</code> , <code>senha_hash</code>             |
| Funções privadas de módulo   | prefixo <code>_</code>                     | <code>_get_db</code> , <code>_carregar_config</code>         |
| Classes                      | <code>PascalCase</code>                    | <code>SCEDSEngine</code> , <code>SCEDSParser</code>          |
| Constantes                   | <code>MAIUSCULO_COM_UNDERSCORE</code>      | <code>TIPOS_VALIDOS</code> , <code>DIAS_JANELA_ALERTA</code> |
| Arquivos de API de módulo    | <code>api_&lt;nome_do_modulo&gt;.py</code> | <code>api_biblioteca.py</code>                               |
| Palavras reservadas do SCEDS | <code>MAIÚSCULO</code> , em português      | <code>CRIAR</code> , <code>BUSCAR</code> , <code>ONDE</code> |

## 8.3. Documentação em Código

Funções públicas devem conter docstring no formato triplo-aspas descrevendo propósito, parâmetros relevantes e valor de retorno, seguindo o padrão já observado nos módulos `core` e `sceds`. O pacote de instalação distribuído para escolas remove comentários de linha do código-fonte para reduzir o tamanho do pacote (ver observação no arquivo `LEIA-ME.md`); a árvore de desenvolvimento mantida pela equipe técnica deve preservar comentários e docstrings integralmente.

## 8.4. Tratamento de Tipos

O código utiliza anotações de tipo do Python (`str | None`, `list[dict]`, etc.) em assinaturas de função sempre que praticável, seguindo a sintaxe disponível a partir do Python 3.10. Módulos novos devem seguir a mesma prática, para que ferramentas de análise estática continuem eficazes.

# Organização dos Módulos

Esta seção resume, para cada módulo funcional, sua finalidade, perfis com acesso e principais entidades de dados manipuladas.

| Módulo              | Perfis com acesso                                                                | Principais tabelas SCEDS                                                                                |
|---------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Agendamento         | professor, secretaria, admin                                                     | reservas                                                                                                |
| Biblioteca          | bibliotecaria, admin                                                             | livros, emprestimos                                                                                     |
| Ocorrências         | coordenadora, admin                                                              | alunos_ocorrencias, ocorrencias                                                                         |
| Secretaria/Portaria | secretaria, portaria, admin                                                      | compromissos, mensagens_chat                                                                            |
| IoT                 | portaria, admin (painel); dispositivos físicos autenticados por token (ingestão) | leituras_iot; estado de portões e AC em arquivos JSON próprios                                          |
| Sinal               | coordenadora, admin                                                              | não utiliza SCEDS — estado mantido em arquivos JSON próprios do módulo e no processo satélite sinal.pyw |
| Monitoramento       | coordenadora, admin                                                              | leitura consolidada de todas as tabelas dos demais módulos, sem tabela própria                          |
| Administração       | admin                                                                            | usuarios; gerencia backups de todas as demais tabelas                                                   |

## 9.1. Independência entre Módulos

Os módulos funcionais não importam código uns dos outros. Toda informação compartilhada entre domínios (por exemplo, o módulo de Monitoramento lendo dados de Agendamento, Biblioteca, Ocorrências e IoT) é obtida por leitura direta das tabelas SCEDS correspondentes, nunca por chamada de função entre módulos. Essa convenção deve ser preservada: um módulo não deve importar o arquivo `api_*.py` de outro módulo.

**AVISO**  
Uma exceção parcial a essa independência é o módulo Ocorrências, cujos scripts de manutenção (`normalizar_tipos.py`) importam funções auxiliares do próprio `api_ocorrencias.py` via inserção manual de caminho no `sys.path`. Trata-se de scripts de manutenção executados manualmente, fora do ciclo de requisições HTTP, e não de acoplamento em tempo de execução do servidor.



# Comunicação entre Componentes

## 10.1. Comunicação Interna (dentro do processo Flask)

A comunicação entre o núcleo e os módulos, e entre os módulos e a camada de persistência, ocorre por chamada direta de função dentro do mesmo processo Python — não há chamadas de rede internas. A única exceção é o módulo Sinal, descrito a seguir.

## 10.2. O Processo Satélite do Módulo Sinal

O módulo Sinal é o único componente do sistema que opera como processo separado do servidor principal. O arquivo `sinal.pyw` é uma aplicação Flask independente, responsável por reproduzir o arquivo de áudio do sinal sonoro localmente na máquina onde está instalado (tipicamente conectada aos alto-falantes da instituição) e por manter a lista de horários programados.

O blueprint `modulos/sinal/api_sinal.py`, executado dentro do processo principal, não acessa o hardware de áudio diretamente; em vez disso, atua como um proxy HTTP, retransmitindo requisições para o processo `sinal.pyw` por meio da função interna `_proxy(metodo, rota, **kwargs)`.

**NOTA**

Essa separação existe porque a reprodução de áudio e a automação de horários do sinal precisam continuar funcionando de forma independente e resiliente, mesmo que o processo web principal seja reiniciado para manutenção. Qualquer alteração no protocolo entre `api_sinal.py` e `sinal.pyw` deve preservar essa independência.

## 10.3. Comunicação com Dispositivos Físicos

A comunicação entre o servidor e os dispositivos ESP32 do módulo IoT ocorre exclusivamente por HTTP simples sobre a rede local, sem uso de MQTT, WebSocket ou outro protocolo de mensageria. Essa escolha simplifica o firmware embarcado, ao custo de uma latência de reação limitada pelo intervalo de polling implementado em cada dispositivo (tipicamente alguns segundos), o que é aceitável para os casos de uso do sistema (abertura de portão, nível de caixa de água, controle de ar-condicionado).

## 10.4. Comunicação com Serviços Externos

O sistema realiza uma única forma de comunicação com serviços externos à rede local: o envio de mensagens de WhatsApp por meio da biblioteca `pywhatkit`, utilizada pelo módulo Sinal para notificar responsáveis sobre saídas antecipadas de alunos. Essa funcionalidade depende de uma sessão de navegador autenticada no WhatsApp Web na máquina onde o processo `sinal.pyw` é executado, e portanto depende de conectividade com a internet no momento do envio.

# Estrutura do Banco de Dados

Cada tabela do SCEDS corresponde a dois arquivos armazenados em `sceds/data/`: um arquivo de esquema (`<tabela>.schema.json`), contendo a definição das colunas, e um arquivo de dados (`<tabela>.sceds`, internamente em formato JSON), contendo os registros e o contador do próximo identificador automático.

## 11.1. Tabelas do Sistema

| Tabela                          | Módulo proprietário    | Finalidade                                                          |
|---------------------------------|------------------------|---------------------------------------------------------------------|
| <code>usuarios</code>           | Núcleo / Administração | Contas de acesso ao sistema e respectivo perfil                     |
| <code>reservas</code>           | Agendamento            | Reservas de recursos compartilhados por data/horário                |
| <code>livros</code>             | Biblioteca             | Catálogo do acervo                                                  |
| <code>emprestimos</code>        | Biblioteca             | Registro de empréstimos e devoluções                                |
| <code>alunos_ocorrencias</code> | Ocorrências            | Cadastro de alunos referenciados no módulo disciplinar              |
| <code>ocorrencias</code>        | Ocorrências            | Registros disciplinares e elogios vinculados a um aluno             |
| <code>compromissos</code>       | Secretaria/Portaria    | Agenda de visitas e compromissos                                    |
| <code>mensagens_chat</code>     | Secretaria/Portaria    | Histórico de mensagens do canal interno entre secretaria e portaria |
| <code>leituras_iot</code>       | IoT                    | Histórico de leituras dos sensores de nível de água                 |

## 11.2. Modelo Físico de Armazenamento

O conteúdo do arquivo de dados de uma tabela segue a estrutura:

```
{
  "registros": [
    { "id": 1, "nome": "...", "...": "..." },
    { "id": 2, "nome": "...", "...": "..." }
  ],
  "proximo_id": 3
}
```

O campo `proximo_id` é incrementado a cada inserção em coluna marcada com os modificadores `CHAVE_PRIMARIA` e `AUTO` simultaneamente, garantindo identificadores inteiros sequenciais e nunca reaproveitados, mesmo após exclusões.

## 11.3. Exemplo de Esquema

```
{
  "tabela": "usuarios",
  "colunas": [
    { "nome": "id",          "tipo": "INTEIRO",  "modificadores": ["CHAVE_PRIMARIA", "AUTO"] },
    { "nome": "nome",       "tipo": "TEXT0",   "modificadores": ["NAO_NULO"] },
    { "nome": "perfil",     "tipo": "TEXT0",   "modificadores": ["NAO_NULO"] },
    { "nome": "senha_hash", "tipo": "TEXT0",   "modificadores": ["NAO_NULO"] },
    { "nome": "ativo",      "tipo": "BOOLEANO", "modificadores": [] },
    { "nome": "criado_em",  "tipo": "DATA_HORA", "modificadores": [] }
  ],
}
```

```
"criada_em": "2026-06-29T20:25:14.999414"
```

```
}
```

## 11.4. Relacionamentos entre Entidades

O SCEDS não implementa chaves estrangeiras nem restrições de integridade referencial. Relacionamentos entre tabelas são convencionais, mantidos por campos que armazenam o identificador da entidade relacionada (por exemplo, `emprestimos.livro_id` referenciando `livros.id`, ou `ocorrencias.aluno_id` referenciando `alunos_ocorrencias.id`). A responsabilidade de manter a consistência desses relacionamentos recai inteiramente sobre o código de cada módulo.

### AVISO

Como não há integridade referencial imposta pelo motor, a exclusão de um registro referenciado por outra tabela (por exemplo, remover um livro com empréstimos associados) não é bloqueada automaticamente pelo SCEDS. Cada módulo deve implementar suas próprias verificações antes de uma exclusão, como observado em `api_ocorrencias.py`, que restringe a remoção de alunos ao perfil admin justamente para mitigar exclusões acidentais que deixariam ocorrências órfãs.

# Manual Completo da Linguagem SCEDS

## 12.1. Histórico e Finalidade

SCEDS — sigla interna do projeto, mantida como nome próprio do subsistema — é a camada de persistência de dados desenvolvida especificamente para o Smart Campus, com dois objetivos combinados: eliminar a dependência de um sistema gerenciador de banco de dados externo, e oferecer, para tarefas de manutenção e depuração, uma notação textual em português capaz de expressar as cinco operações fundamentais de manipulação de dados (criação de tabela, inserção, busca, atualização e exclusão) sem que o mantenedor precise escrever chamadas Python diretamente.

NOTA SOBRE O ESCOPO DA LINGUAGEM

É importante que qualquer equipe futura compreenda com precisão o escopo do SCEDS: trata-se de uma **notação de comandos para operações de dados**, análoga em finalidade — ainda que muito mais restrita em recursos — a um subconjunto de SQL, e não de uma linguagem de programação de propósito geral. O SCEDS, na versão 1.0.0 aqui documentada, **não possui** variáveis com escopo, estruturas condicionais, estruturas de repetição, declaração de funções pelo usuário, transações multi-instrução, índices, ou controle de concorrência além do bloqueio por tabela descrito na Seção 12.5. Este manual documenta com exatidão o que existe na implementação atual; qualquer descrição de recurso não implementado, encontrada em versões anteriores de outros documentos, deve ser desconsiderada até que o recurso seja efetivamente construído e testado.

## 12.2. Arquitetura Interna

O subsistema SCEDS é composto por três arquivos-fonte na pasta `sceds/`, com responsabilidades estritamente separadas:

| Arquivo                   | Classe                   | Responsabilidade                                                                                                                                                                                                                                           |
|---------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>parser.py</code>    | <code>SCEDSParser</code> | Converte um comando textual em português em uma estrutura de dados intermediária (uma árvore de sintaxe simplificada, representada como <code>dict</code> Python)                                                                                          |
| <code>engine.py</code>    | <code>SCEDSEngine</code> | Executa operações de leitura e escrita sobre os arquivos <code>.sceds</code> e <code>.schema.json</code> , aplicando validação de tipos e bloqueio de concorrência por tabela                                                                              |
| <code>connector.py</code> | <code>SCEDS</code>       | Classe de fachada (facade) que unifica o acesso via linguagem textual (método <code>executar()</code> ) e o acesso via métodos Python diretos ( <code>buscar</code> , <code>inserir</code> , <code>atualizar</code> , <code>deletar</code> , entre outros) |

Na prática operacional do sistema, a quase totalidade dos módulos utiliza a classe `SCEDS` por meio de seus métodos Python diretos, e não por meio de comandos textuais interpretados pelo parser. O método `executar()` — e, por consequência, a linguagem textual descrita neste capítulo — destina-se sobretudo a scripts de manutenção, depuração manual e uso interativo por parte da equipe técnica.

## 12.3. Funcionamento do Interpretador (visão geral)

O método `SCEDS.executar(comando: str)` encaminha o texto recebido ao `SCEDSParser.parse()`, que devolve um dicionário com a chave `"operacao"` indicando qual das cinco operações foi reconhecida, além dos parâmetros específicos dessa operação (tabela, colunas, dados, condições). O `connector.py` então despacha essa estrutura para o método correspondente de `SCEDSEngine`. Não há uma etapa de compilação, otimização de consulta, ou geração de código intermediário além dessa tradução direta.

## 12.4. Tipos de Dados

O SCEDS define seis tipos de coluna, verificados em tempo de criação de tabela e aplicados na conversão de valores em cada inserção ou atualização:

| Tipo      | Representação Python subjacente        | Observações de conversão                                                                                                                                                                                             |
|-----------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INTEIRO   | int                                    | Conversão via <code>int(valor)</code> ; falha silenciosa retorna o valor original sem conversão                                                                                                                      |
| TEXT0     | str                                    | Qualquer valor é convertido por <code>str(valor)</code> quando não corresponde a nenhum outro tipo                                                                                                                   |
| DECIMAL   | float                                  | Conversão via <code>float(valor)</code>                                                                                                                                                                              |
| BOOLEANO  | bool                                   | Aceita, além de <code>True</code> / <code>False</code> literais, as strings <code>"verdadeiro"</code> , <code>"true"</code> , <code>"1"</code> , <code>"sim"</code> (demais valores resultam em <code>False</code> ) |
| DATA      | str em formato ISO 8601 ( AAAA-MM-DD ) | Objetos <code>date</code> do Python são convertidos via <code>.isoformat()</code> ; strings são armazenadas como recebidas                                                                                           |
| DATA_HORA | str em formato ISO 8601 completo       | Mesma lógica de <code>DATA</code> , aplicada a objetos <code>datetime</code>                                                                                                                                         |

### AVISO

A conversão de tipos do SCEDS não é estritamente validada: em caso de exceção durante a conversão (por exemplo, tentar converter o texto `"abc"` para `INTEIRO` ), o método `_converter_valor()` captura a exceção silenciosamente e devolve o valor original, sem sinalizar erro ao chamador. Módulos que inserem dados de origem externa (formulários web, dispositivos IoT) devem validar os tipos antes de chamar o SCEDS, não devendo depender do motor para rejeitar valores incompatíveis.

## 12.5. Concorrência e Integridade

Cada tabela possui um `threading.Lock` próprio, criado sob demanda e mantido em memória pelo `SCEDSEngine` durante a vida do processo. Toda operação de escrita (inserção, atualização, exclusão) adquire esse bloqueio antes de ler o arquivo, modificar seu conteúdo em memória e regravá-lo por completo, liberando o bloqueio ao final. Esse mecanismo garante que duas requisições concorrentes sobre a mesma tabela, dentro do mesmo processo, não corrompam o arquivo por escrita simultânea.

### AVISO DE ESCALABILIDADE

O bloqueio é local ao processo Python em execução. Caso o sistema seja, no futuro, executado como múltiplos processos ou múltiplas instâncias (por exemplo, sob um servidor WSGI com vários workers), essa proteção deixa de ser suficiente, pois cada processo manteria seus próprios bloqueios independentes. A versão atual do sistema pressupõe execução como processo único ( `app.run(threaded=True)` ), e essa premissa deve ser preservada ou revista de forma coordenada com o SCEDS caso a topologia de execução mude.

O motor não implementa transações que abranjam múltiplas operações, nem capacidade de reversão (rollback). Cada chamada a `inserir` , `atualizar` ou `deletar` é atômica isoladamente (todo o arquivo é regravado de uma só vez), mas sequências de operações relacionadas executadas por um módulo (por exemplo, debitar um recurso e registrar um log) não são atômicas em conjunto.

# Sintaxe da Linguagem SCEDS

A sintaxe do SCEDS é definida por expressões regulares aplicadas ao texto do comando após normalização (remoção de linhas de comentário iniciadas por #, união de múltiplas linhas em uma única linha e colapso de espaços consecutivos). Não há um analisador léxico (tokenizador) formal separado da análise sintática: cada operação possui seu próprio método de reconhecimento por expressão regular em SCEDSParser.

## 13.1. Estrutura Geral de um Comando

```
<PALAVRA-CHAVE DA OPERAÇÃO> ... argumentos específicos da operação ...
```

A primeira palavra do comando (após normalização de espaços) determina a operação a ser executada. As cinco palavras-chave de operação reconhecidas são CRIAR, INSERIR, BUSCAR, ATUALIZAR e DELETAR, em qualquer combinação de maiúsculas e minúsculas (a verificação da primeira palavra é sensível a caixa alta, pois o parser aplica .upper() antes de comparar).

## 13.2. Comentários

Linhas iniciadas por # são descartadas antes da análise sintática, permitindo comentar blocos de comandos SCEDS mantidos em scripts de manutenção.

## 13.3. Literais

| Literal                   | Sintaxe                      | Exemplo     |
|---------------------------|------------------------------|-------------|
| Texto                     | Delimitado por aspas simples | 'professor' |
| Número inteiro ou decimal | Sem aspas                    | 42, 3.5     |
| Booleano verdadeiro       | verdadeiro ou true           | —           |
| Booleano falso            | falso ou false               | —           |
| Nulo                      | nulo ou null                 | —           |

## 13.4. Sintaxe por Operação

```
CRIAR TABELA nome_tabela (  
    campo1 TIPO [modificador ...],  
    campo2 TIPO [modificador ...],  
    ...  
)  
  
INSERIR EM nome_tabela (campo1, campo2, ...) VALORES (valor1, valor2, ...)  
  
BUSCAR * DE nome_tabela [ONDE campo = valor [E campo = valor ...]]  
  
ATUALIZAR nome_tabela DEFINIR campo = valor [, campo = valor ...] ONDE campo = valor [E campo = valor ...]  
  
DELETAR DE nome_tabela ONDE campo = valor [E campo = valor ...]
```

A cláusula `ONDE` aceita apenas o operador de igualdade, e apenas conjunções por `E` (equivalente lógico a `AND`). Não há suporte sintático a `OU` (OR), negação, operadores relacionais (maior, menor) ou agrupamento por parênteses dentro da cláusula `ONDE`.

# Gramática Formal

A gramática a seguir descreve, em notação simplificada semelhante a BNF, a linguagem efetivamente reconhecida pelo `SCEDSParser`. Ela é apresentada como referência de consulta; a implementação real não utiliza um gerador de parser a partir de gramática formal, mas expressões regulares equivalentes ao que se segue.

```
comando          ::= criar_tabela | inserir | buscar | atualizar | deletar

criar_tabela     ::= "CRIAR" "TABELA" identificador "(" lista_colunas ")"
lista_colunas    ::= definicao_coluna ( "," definicao_coluna )*
definicao_coluna ::= identificador tipo ( modificador )*
tipo             ::= "INTEIRO" | "TEXTO" | "DECIMAL" | "BOOLEANO" | "DATA" | "DATA_HORA"
modificador      ::= "CHAVE_PRIMARIA" | "AUTO" | "NAO_NULO" | "UNICO"
                  | "PADRAO" literal

inserir          ::= "INSERIR" "EM" identificador
                  "(" lista_identificadores ")"
                  "VALORES" "(" lista_literais ")"

buscar           ::= "BUSCAR" "*" "DE" identificador ( clausula_onde )?

atualizar        ::= "ATUALIZAR" identificador "DEFINIR" lista_atribuicoes
                  clausula_onde

deletar          ::= "DELETAR" "DE" identificador clausula_onde

clausula_onde    ::= "ONDE" condicao ( "E" condicao )*
condicao          ::= identificador "=" literal
lista_atribuicoes ::= atribuicao ( "," atribuicao )*
atribuicao        ::= identificador "=" literal

literal          ::= texto_aspas_simples | numero | booleano | nulo
identificador    ::= [A-Za-z_][A-Za-z0-9_]*
lista_identificadores ::= identificador ( "," identificador )*
lista_literais   ::= literal ( "," literal )*
```

## NOTA DE IMPLEMENTAÇÃO

A divisão da lista de colunas em `CRIAR TABELA` respeita parênteses aninhados (função `_dividir_colunas()`), permitindo que um modificador como `PADRAO` receba, em tese, um valor entre parênteses — embora nenhuma tabela do sistema, na versão atual, utilize essa possibilidade.



# Tipos de Dados (Referência Detalhada)

Este capítulo complementa a Seção 12.4 com o comportamento detalhado de cada tipo em face de valores ausentes, valores padrão e modificadores de coluna.

## 15.1. Modificadores de Coluna

| Modificador    | Efeito                                                                                                                                                                                                                          |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CHAVE_PRIMARIA | Identifica a coluna como identificador único lógico do registro. Na implementação atual, produz efeito prático apenas quando combinado com <code>AUTO</code> .                                                                  |
| AUTO           | Quando combinado com <code>CHAVE_PRIMARIA</code> , o valor da coluna é atribuído automaticamente pelo motor a partir do contador <code>proximo_id</code> da tabela, sendo ignorado se fornecido explicitamente em uma inserção. |
| NAO_NULO       | Em inserções, exige que a coluna receba um valor; na ausência de valor, verifica se há um modificador <code>PADRAO:</code> associado antes de lançar <code>ValueError</code> .                                                  |
| UNICO          | Ao inserir, percorre todos os registros já existentes da tabela e rejeita a inserção caso o valor já esteja em uso em outro registro.                                                                                           |
| PADRAO valor   | Fornece um valor padrão de texto, utilizado apenas quando a coluna também é <code>NAO_NULO</code> e nenhum valor foi informado na inserção.                                                                                     |

## 15.2. Comportamento em Atualizações

O modo `"atualizar"` de `_validar_e_preparar()` não aplica a verificação de `NAO_NULO` nem a geração automática de valores `AUTO` — apenas converte os tipos dos campos efetivamente fornecidos em `novos_dados`. Isso significa que uma atualização parcial (apenas alguns campos) é suportada nativamente, mas também que a obrigatoriedade de campos não-nulos não é reforçada em operações de atualização, apenas em inserções.

## 15.3. Ausência de Tipos Compostos

O SCEDS não oferece tipos de coluna para listas, objetos aninhados ou referências diretas a outras tabelas. Quando um módulo precisa associar múltiplos valores a um único registro (por exemplo, múltiplos itens de uma reserva), a convenção observada no código é utilizar uma coluna `TEXT0` contendo uma string serializada em JSON, com a responsabilidade de serialização e desserialização atribuída ao módulo chamador, não ao motor SCEDS.

# Operadores

O conjunto de operadores da linguagem SCEDS é deliberadamente mínimo, refletindo seu propósito de camada de persistência e não de linguagem de propósito geral.

| Operador                     | Símbolo | Contexto de uso                           | Observações                                                                                                                                                                                       |
|------------------------------|---------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Igualdade                    | =       | Cláusulas ONDE e DEFINIR                  | Único operador de comparação disponível; não há !=, >, <, >=, <= ou LIKE                                                                                                                          |
| Conjunção lógica             | E       | Combinação de múltiplas condições em ONDE | Equivalente a AND; não há operador OU nem NÃO (NOT)                                                                                                                                               |
| Separador de lista           | ,       | Listas de colunas, valores e atribuições  | Não é, tecnicamente, um operador da linguagem de consulta, mas um delimitador sintático                                                                                                           |
| Asterisco (todas as colunas) | *       | Comando BUSCAR * DE ...                   | É o único valor aceito nessa posição; não há suporte a seleção de colunas específicas na sintaxe textual (essa seleção é feita programaticamente pelo chamador, filtrando o dicionário retornado) |

### NOTA

Operações aritméticas, concatenação de texto e funções de agregação (soma, contagem, média) não fazem parte da sintaxe textual do SCEDS. Quando um módulo precisa desses cálculos (por exemplo, o módulo de Monitoramento), eles são realizados em código Python após a recuperação dos registros brutos via buscar(), e não dentro da linguagem SCEDS.

# Comandos da Linguagem

Esta seção documenta cada um dos cinco comandos do SCEDS no formato de ficha técnica padronizada: descrição, sintaxe completa, parâmetros, valor de retorno, limitações e um exemplo comentado.

CRIAR TABELA

**Descrição:** Cria uma nova tabela, gravando seu arquivo de esquema e seu arquivo de dados vazio.

**Sintaxe:**

```
CRIAR TABELA nome_tabela (  
    campo TIPO [modificadores...],  
    ...  
)
```

**Parâmetros:** nome da tabela (identificador); lista de definições de coluna, cada uma com nome, tipo e modificadores opcionais.

**Retorno:** `None`. A operação não produz valor de retorno utilizável pelo chamador.

**Limitações:** lança `ValueError` se a tabela já existir, ou se algum tipo de coluna informado não pertencer ao conjunto de tipos válidos (Capítulo 15). Não há suporte a alteração de esquema (equivalente a `ALTER TABLE`) nem a criação condicional (equivalente a `CREATE TABLE IF NOT EXISTS`).

**Exemplo:**

```
CRIAR TABELA usuarios (  
    id INTEIRO CHAVE_PRIMARIA AUTO,  
    nome TEXTO NAO_NULO,  
    perfil TEXTO NAO_NULO,  
    senha_hash TEXTO NAO_NULO,  
    ativo BOOLEANO,  
    criado_em DATA_HORA  
)
```

INSERIR EM

**Descrição:** Insere um novo registro em uma tabela existente.

**Sintaxe:**

```
INSERIR EM tabela (campo1, campo2, ...) VALORES (valor1, valor2, ...)
```

**Parâmetros:** nome da tabela; lista de nomes de campo; lista de valores literais, na mesma ordem e quantidade dos campos.

**Retorno:** dicionário Python com o registro completo já inserido, incluindo valores gerados automaticamente (como o identificador `AUTO`).

**Limitações:** lança `SyntaxError` se a quantidade de campos e valores não coincidir; lança `ValueError` se um campo `NAO_NULO` sem valor padrão não for informado, ou se um valor colidir com um campo marcado como `UNICO`.

**Exemplo:**

```
INSERIR EM usuarios (nome, perfil, senha_hash, ativo)
VALORES ('Ana Souza', 'bibliotecaria', '$2b$12$abc...', verdadeiro)
```

## BUSCAR

**Descrição:** Recupera os registros de uma tabela que satisfazem, opcionalmente, uma cláusula de condições.

**Sintaxe:**

```
BUSCAR * DE tabela [ONDE campo = valor [E campo = valor ...]]
```

**Parâmetros:** nome da tabela; cláusula `ONDE` opcional, com uma ou mais condições de igualdade unidas por `E`.

**Retorno:** lista de dicionários (uma cópia independente de cada registro correspondente; alterações no dicionário retornado não afetam os dados armazenados).

**Limitações:** não há suporte a ordenação, paginação ou seleção de colunas específicas por meio da sintaxe textual — esses recursos existem apenas nos métodos Python diretos (`ordenar_por`, `limite` em `SCEDS.buscar()`), não no comando textual `BUSCAR`.

**Exemplo:**

```
BUSCAR * DE usuarios ONDE perfil = 'professor' E ativo = verdadeiro
```

## ATUALIZAR

**Descrição:** Modifica os registros de uma tabela que satisfazem a cláusula de condições, atribuindo novos valores aos campos indicados.

**Sintaxe:**

```
ATUALIZAR tabela DEFINIR campo = valor [, campo = valor ...] ONDE campo = valor [E campo = valor ...]
```

**Parâmetros:** nome da tabela; lista de atribuições (`DEFINIR`); cláusula `ONDE`, obrigatória nesta operação.

**Retorno:** número inteiro de registros efetivamente atualizados.

**Limitações:** a cláusula `ONDE` é obrigatória na sintaxe textual — não é possível atualizar todos os registros de uma tabela sem condição por meio do comando textual (essa restrição não existe no método Python `SCEDS.atualizar()`, que aceita um dicionário de condições vazio).

**Exemplo:**

```
ATUALIZAR usuarios DEFINIR ativo = falso ONDE id = 12
```

## DELETAR

**Descrição:** Remove permanentemente os registros de uma tabela que satisfazem a cláusula de condições.

**Sintaxe:**

```
DELETAR DE tabela ONDE campo = valor [E campo = valor ...]
```

**Parâmetros:** nome da tabela; cláusula `ONDE`, obrigatória.

**Retorno:** número inteiro de registros removidos.

**Limitações:** não há exclusão reversível (não há "lixeira" ou marcação lógica de exclusão no motor — essa responsabilidade, quando desejada, cabe ao módulo, como no padrão de campo `ativo` observado na tabela `usuarios`, preferido no lugar de exclusão física para desativação de contas).

**Exemplo:**

```
DELETAR DE reservas ONDE id = 87
```

# Palavras Reservadas

| Palavra-chave                                                                                                                         | Categoria             | Uso                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------|------------------------------------------------------|
| <code>CRIAR</code>                                                                                                                    | Operação              | Inicia a criação de uma tabela                       |
| <code>TABELA</code>                                                                                                                   | Estrutural            | Acompanha <code>CRIAR</code>                         |
| <code>INSERIR</code>                                                                                                                  | Operação              | Inicia a inserção de um registro                     |
| <code>EM</code>                                                                                                                       | Estrutural            | Acompanha <code>INSERIR</code>                       |
| <code>VALORES</code>                                                                                                                  | Estrutural            | Introduz a lista de valores de uma inserção          |
| <code>BUSCAR</code>                                                                                                                   | Operação              | Inicia uma consulta                                  |
| <code>DE</code>                                                                                                                       | Estrutural            | Acompanha <code>BUSCAR</code> e <code>DELETAR</code> |
| <code>ONDE</code>                                                                                                                     | Estrutural            | Introduz a cláusula de condições                     |
| <code>E</code>                                                                                                                        | Operador lógico       | Conjunção entre condições                            |
| <code>ATUALIZAR</code>                                                                                                                | Operação              | Inicia uma atualização                               |
| <code>DEFINIR</code>                                                                                                                  | Estrutural            | Introduz a lista de atribuições de uma atualização   |
| <code>DELETAR</code>                                                                                                                  | Operação              | Inicia uma exclusão                                  |
| <code>INTEIRO</code> , <code>TEXTO</code> , <code>DECIMAL</code> , <code>BOOLEANO</code> , <code>DATA</code> , <code>DATA_HORA</code> | Tipo de dado          | Usadas na definição de colunas                       |
| <code>CHAVE_PRIMARIA</code> , <code>AUTO</code> , <code>NAO_NULO</code> , <code>UNICO</code> , <code>PADRAO</code>                    | Modificador de coluna | Usadas na definição de colunas                       |
| <code>verdadeiro</code> , <code>true</code> , <code>falso</code> , <code>false</code> , <code>nulo</code> , <code>null</code>         | Literal               | Valores literais especiais                           |

## NOTA

O reconhecimento das palavras-chave estruturais ( `TABELA`, `EM`, `DE`, `VALORES`, `ONDE`, `DEFINIR` ) ocorre embutido nas expressões regulares de cada operação, e não por meio de uma lista centralizada de palavras reservadas no código. Ao propor novos comandos, a equipe de manutenção deve verificar manualmente que os novos identificadores de tabela ou campo escolhidos pelos módulos não coincidam com nenhuma dessas palavras, pois o parser não realiza esse tipo de verificação.

# Funções Internas

O SCEDS não expõe, na linguagem textual, um conjunto de funções internas invocáveis pelo usuário (como funções de data, matemáticas ou de string disponíveis em SQL). A camada de fachada ( `connector.py` ) oferece, no entanto, um conjunto de métodos Python auxiliares que complementam as cinco operações da linguagem e que devem ser considerados parte da interface pública do SCEDS para fins de manutenção:

| Método                                      | Finalidade                                                                                                                            |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>buscar_um(tabela, onde)</code>        | Retorna o primeiro registro correspondente, ou <code>None</code> ; conveniência sobre <code>buscar()</code> com <code>limite=1</code> |
| <code>contar(tabela, onde=None)</code>      | Retorna a quantidade de registros correspondentes, sem materializar seleção de colunas adicionais                                     |
| <code>tabela_existe(tabela)</code>          | Verifica a existência do arquivo de dados da tabela                                                                                   |
| <code>listar_tabelas()</code>               | Enumera todas as tabelas presentes na pasta de dados                                                                                  |
| <code>dropar_tabela(tabela)</code>          | Remove definitivamente o arquivo de dados e o arquivo de esquema de uma tabela                                                        |
| <code>backup_tabela(tabela, destino)</code> | Copia o arquivo <code>.sceds</code> de uma tabela para uma pasta de destino                                                           |
| <code>backup_completo(destino)</code>       | Executa <code>backup_tabela</code> para todas as tabelas existentes                                                                   |

### NOTA

Nenhum desses métodos possui equivalente na sintaxe textual da linguagem SCEDS; são acessíveis apenas via chamada Python direta a uma instância da classe `SCEDS` . Qualquer necessidade futura de expor essas capacidades também por meio de comandos textuais exigirá extensão do `SCEDSParser` , hoje limitado às cinco operações descritas no Capítulo 17.

# Sistema de Execução

Este capítulo descreve o comportamento em tempo de execução do sistema como um todo, complementando a documentação já apresentada especificamente para o SCEDS.

## 20.1. Modelo de Processo

O sistema executa como um único processo Python de longa duração, iniciado por `python app.py`, com o servidor de desenvolvimento do Flask configurado em modo `threaded=True` e `debug=False`. Cada requisição HTTP é atendida em uma thread própria dentro desse processo; não há múltiplos processos trabalhadores (workers), filas de tarefas em segundo plano além da thread única do agendador de automações IoT, nem execução distribuída.

**AVISO**

O uso do servidor de desenvolvimento embutido do Flask (`app.run(...)`) para atender ao tráfego de produção da instituição é uma decisão consciente de simplicidade operacional, adequada à carga esperada (uma única instituição, poucas dezenas de usuários concorrentes). Caso a carga de uso cresça significativamente, a migração para um servidor WSGI de produção (Waitress é a opção mais compatível com Windows, dado o SO de referência do sistema) deve ser avaliada — o que, por sua vez, exige reavaliar a estratégia de bloqueio por processo do SCEDS, descrita na Seção 12.5.

## 20.2. Threads de Segundo Plano

O módulo IoT inicia, na importação de `api_iot.py`, uma thread daemon (`_loop_automacoes`) que permanece em execução por toda a vida do processo, verificando a cada vinte segundos se algum horário de automação programada foi atingido. A verificação de que a thread já foi iniciada (`hasattr(blueprint, "_scheduler_iniciado")`) evita duplicação da thread em caso de reimportação do módulo.

## 20.3. Encerramento do Sistema

O encerramento do processo principal (por exemplo, via `Ctrl+C` no terminal, ou pelo gerenciador de serviços do Windows quando configurado para iniciar automaticamente) interrompe imediatamente todas as threads, inclusive o agendador de automações IoT. Não há rotina de finalização (shutdown hook) que persista estado pendente, pois o SCEDS grava cada alteração de dados imediatamente após a operação correspondente — não há, portanto, dados em memória aguardando gravação no momento do encerramento, exceto estados momentâneos de dispositivos IoT mantidos em arquivos JSON próprios do módulo, também gravados de forma síncrona a cada alteração.



# Interpretador Desenvolvido em Python

Este capítulo apresenta o interpretador da linguagem SCEDS ( `SCEDSParser` ) em nível de implementação, complementando a visão arquitetural do Capítulo 12.

## 21.1. Ponto de Entrada

O método público `parse(self, comando: str) -> dict` é o único ponto de entrada do interpretador. Ele normaliza o texto recebido (remoção de comentários, união de linhas, colapso de espaços), identifica a primeira palavra em maiúsculas e despacha para um dos cinco métodos privados de reconhecimento ( `_parse_criar` , `_parse_inserir` , `_parse_buscar` , `_parse_atualizar` , `_parse_deletar` ), lançando `SyntaxError` quando a primeira palavra não corresponde a nenhuma operação conhecida.

## 21.2. Estratégia de Reconhecimento

Cada método de reconhecimento aplica uma expressão regular única, com âncoras de início e fim de string, sobre o comando já normalizado em uma única linha. Não há retrocesso (backtracking) sintático entre operações: se a expressão regular da operação identificada pela primeira palavra não casar com o restante do comando, o interpretador lança `SyntaxError` imediatamente, sem tentar reinterpretar o comando como outra operação.

## 21.3. Funções Auxiliares de Análise

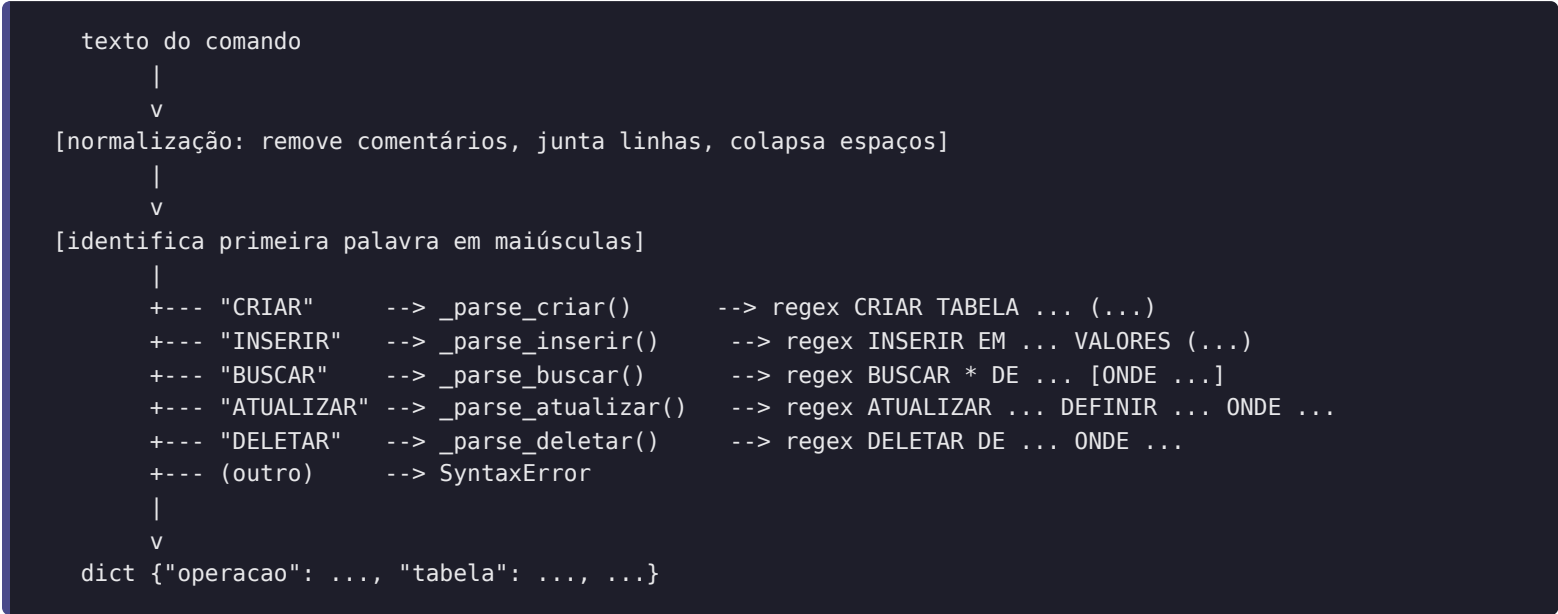
| Função                                   | Finalidade                                                                                                                                        |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>_dividir_colunas(corpo)</code>     | Separa a lista de definições de coluna por vírgula, respeitando parênteses aninhados                                                              |
| <code>_parse_coluna(definicao)</code>    | Interpreta uma definição individual de coluna (nome, tipo, modificadores)                                                                         |
| <code>_parse_lista_valores(texto)</code> | Separa uma lista de valores literais por vírgula, respeitando strings entre aspas simples que podem conter vírgulas internas                      |
| <code>_converter_literal(texto)</code>   | Converte um literal textual (fora de aspas) para o tipo Python mais provável: booleano, nulo, inteiro, decimal ou texto, nessa ordem de tentativa |
| <code>_parse_condicoes(texto)</code>     | Interpreta a cláusula <code>ONDE</code> , dividindo por <code>E</code> e cada condição por <code>=</code>                                         |
| <code>_parse_atribuicoes(texto)</code>   | Interpreta a cláusula <code>DEFINIR</code> , de forma análoga a <code>_parse_condicoes</code>                                                     |

## 21.4. Saída do Interpretador

A saída de `parse()` é sempre um dicionário contendo, no mínimo, as chaves `"operacao"` e `"tabela"` , acrescido das chaves específicas de cada operação ( `"colunas"` , `"dados"` , `"onde"` , `"novos_dados"` , conforme o caso). Essa estrutura é consumida exclusivamente por `SCEDS.executar()` , em `connector.py` .

# Arquitetura do Parser

## 22.1. Diagrama de Fluxo de Análise



## 22.2. Ausência de Árvore de Sintaxe Abstrata (AST) Genérica

Diferentemente de interpretadores de linguagens de propósito geral, o SCEDSParser não constrói uma árvore de sintaxe abstrata genérica e recursiva. Cada operação produz uma estrutura plana (um único nível de dicionário, eventualmente com uma lista de sub-dicionários para colunas ou condições), suficiente para as cinco operações suportadas. Essa escolha simplifica a implementação, mas limita a extensibilidade: qualquer comando composto (por exemplo, uma futura cláusula de sub-consulta) exigiria uma reformulação da estratégia de análise, provavelmente substituindo as expressões regulares por um tokenizador e um parser recursivo genuíno.

## 22.3. Extensibilidade Recomendada

Para equipes que avaliarem estender a sintaxe do SCEDS (por exemplo, adicionar o operador lógico `OU`, ou operadores relacionais na cláusula `ONDE`), recomenda-se:

1. Preservar a compatibilidade retroativa das cinco operações e da sintaxe já documentada nos Capítulos 13 e 17, dado que scripts de manutenção existentes dependem dela.
2. Concentrar qualquer extensão de análise de condições em `_parse_condicoes()` e `_parse_atribuicoes()`, que já isolam essa responsabilidade das demais.
3. Avaliar a substituição da abordagem por expressão regular única por um tokenizador explícito, caso a complexidade da cláusula `ONDE` cresça além de conjunções simples.

# Funcionamento do Compilador/Interpretador

O SCEDS opera exclusivamente por interpretação direta: não há etapa de compilação prévia, geração de bytecode, cache de plano de execução ou qualquer forma de pré-processamento persistente de comandos. Cada chamada a `executar()` repete integralmente o ciclo de análise sintática e execução, do texto bruto ao resultado final.

## 23.1. Ciclo Completo de uma Chamada `executar()`

```
SCEDS.executar(comando)
├── 1. SCEDSParser.parse(comando)           → dict (operação + parâmetros)
├── 2. despacho por ast["operacao"]:
│   ├── "criar"      → SCEDSEngine.criar_tabela(tabela, colunas)
│   ├── "inserir"    → SCEDSEngine.inserir(tabela, dados)
│   ├── "buscar"     → SCEDSEngine.buscar(tabela, onde)
│   ├── "atualizar"  → SCEDSEngine.atualizar(tabela, novos_dados, onde)
│   └── "deletar"    → SCEDSEngine.deletar(tabela, onde)
└── 3. SCEDSEngine adquire lock da tabela (quando aplicável),
    lê o arquivo .sceds correspondente, aplica a operação
    em memória, regrava o arquivo por completo, libera o lock
    e devolve o resultado ao chamador.
```

## 23.2. Custo Computacional

Cada operação de leitura (`BUSCAR`) carrega o arquivo `.sceds` inteiro da tabela em memória e filtra os registros em Python puro, com custo proporcional ao número total de registros da tabela, independentemente de quantos satisfaçam a condição. Não há índices, portanto não há caminho de acesso mais eficiente que a leitura completa. Para os volumes de dados típicos de uma única instituição de ensino (tipicamente milhares, não milhões, de registros por tabela), esse custo é aceitável; equipes futuras que observem degradação de desempenho devem tratá-la primeiro como um sinal para introduzir índices ou paginação, e não como defeito a ser corrigido por reescrita completa do motor.

## 23.3. Ausência de Plano de Consulta

Por não haver otimizador nem índice, não existe conceito de "plano de execução" no SCEDS: toda consulta é executada por varredura completa (*full scan*) do arquivo de dados. Este é um limite estrutural conhecido e documentado da arquitetura, não uma falha de implementação a ser corrigida isoladamente — qualquer melhoria de desempenho substancial implicaria redesenho do formato de armazenamento.

# Tratamento de Erros

## 24.1. Exceções da Camada SCEDS

| Exceção           | Origem      | Situação                                                                                                                                 |
|-------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------|
| SyntaxError       | SCEDSParser | Comando textual não corresponde à sintaxe de nenhuma operação reconhecida, ou não respeita o formato específico da operação identificada |
| ValueError        | SCEDSEngine | Tabela já existente em CRIAR TABELA ; tipo de coluna inválido; campo obrigatório ausente; valor duplicado em coluna UNICO                |
| FileNotFoundError | SCEDSEngine | Tentativa de operar sobre tabela ou esquema inexistente                                                                                  |

Nenhuma dessas exceções é capturada dentro da própria camada SCEDS; a responsabilidade de tratamento cabe ao módulo chamador, tipicamente em um bloco try/except na rota Flask correspondente, convertendo a exceção em uma resposta JSON de erro conforme o padrão descrito na Seção 7.4.

## 24.2. Tratamento de Erros HTTP

O núcleo define três manipuladores de erro globais em app.py , cobrindo os códigos 403 (acesso negado), 404 (página não encontrada) e 500 (erro interno), todos renderizando o template comum erro.html com título e mensagem apropriados. O manipulador de erro 500 adicionalmente registra a exceção no log do servidor antes de responder.

## 24.3. Falha Isolada de Módulo

Conforme descrito na Seção 3.3, falhas de importação de um módulo individual durante o registro de blueprints ( core/router.py ) são capturadas e registradas em log, sem interromper a inicialização dos demais módulos nem do servidor como um todo. Essa é uma decisão deliberada de resiliência: um módulo com erro de sintaxe ou dependência ausente não deve tornar o sistema inteiro indisponível.

### AVISO

Essa resiliência tem custo: um módulo que falhou ao ser registrado permanece silenciosamente inacessível (suas rotas simplesmente não existem, resultando em 404 para qualquer usuário que tente acessá-lo), a menos que o log do servidor seja consultado. Após qualquer atualização de código, a equipe de manutenção deve verificar logs/servidor.log à procura de mensagens "não pôde ser importado" antes de considerar a atualização concluída com sucesso.

# Logs

## 25.1. Arquivos de Log

| Arquivo             | Origem                          | Conteúdo                                                            |
|---------------------|---------------------------------|---------------------------------------------------------------------|
| logs/servidor.log   | Processo principal ( app.py )   | Eventos de autenticação, registro de módulos, erros internos        |
| logs/sinal.log      | Processo satélite ( sinal.pyw ) | Eventos de acionamento do sinal sonoro e automações de horário      |
| logs/instalacao.txt | Instalador ( instalar.py )      | Relatório de instalação, incluindo a senha inicial do administrador |

## 25.2. Configuração de Log do Processo Principal

O log do processo principal é configurado em `app.py` por meio de `logging.basicConfig()`, com nível `INFO`, formato incluindo data/hora e nível, e dois manipuladores simultâneos: gravação em arquivo (`logs/servidor.log`) e exibição no console (`StreamHandler`), útil para acompanhamento em tempo real quando o servidor é executado em um terminal visível.

## 25.3. Convenção de Prefixo por Componente

Mensagens de log seguem a convenção de prefixar a origem entre colchetes, como em `[Auth]`, `[Router]` e `[500]`, facilitando a filtragem manual do arquivo de log durante uma investigação. Novos módulos devem adotar a mesma convenção, utilizando o nome do módulo como prefixo.

**AVISO DE SEGURANÇA**

O log de autenticação registra tentativas de login malsucedidas, incluindo o endereço IP de origem, mas não registra a senha informada — apenas o fato da falha. Nenhuma alteração futura ao módulo de autenticação deve introduzir o registro de senhas, mesmo malsucedidas, em texto claro nos arquivos de log.

# Sistema de Configuração

## 26.1. Arquivo de Configuração Global

O arquivo `core/config.json` concentra os parâmetros globais da instalação, gerados originalmente pelo instalador e passíveis de edição manual posterior pela equipe técnica:

| Campo                                                                             | Finalidade                                                                                                                                                                   |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>caminho_base</code>                                                         | Caminho absoluto da pasta de instalação ( <code>SmartCampus/</code> ), usado por todos os módulos para localizar a pasta de dados do SCEDS e o restante da árvore do projeto |
| <code>porta_api</code>                                                            | Porta TCP em que o servidor Flask escuta                                                                                                                                     |
| <code>host_api</code>                                                             | Endereço de bind do servidor ( <code>0.0.0.0</code> por padrão, ouvindo em todas as interfaces de rede)                                                                      |
| <code>encoding</code>                                                             | Codificação de texto padrão utilizada na leitura/escrita de arquivos                                                                                                         |
| <code>versao</code>                                                               | Versão do sistema instalada, exibida no endpoint de status e nos relatórios                                                                                                  |
| <code>nome_escola</code> , <code>sigla_escola</code> , <code>bairro_cidade</code> | Identidade da instituição, exibida nos templates por meio do contexto global injetado em <code>app.py</code>                                                                 |

## 26.2. Arquivos de Configuração Específicos de Módulo

Além do arquivo global, módulos individuais mantêm seus próprios arquivos de configuração em formato JSON, quando os dados envolvidos são estruturais e não fazem sentido como registros de tabela SCEDS: `modulos/agendamento/recursos.json` (catálogo de recursos reserváveis), `modulos/iot/dispositivos.json` (tokens e parâmetros de automação dos dispositivos físicos) e `modulos/ocorrencias/turmas.json` (lista fixa de turmas da instituição).

**NOTA**

Esses arquivos são gerados interativamente pelo instalador ( `instalar.py` ) no momento da implantação inicial, mas podem — e frequentemente precisam — ser editados manualmente ao longo da vida do sistema, por exemplo quando a instituição reorganiza suas turmas em um novo ano letivo (ver `verificar_turmas.py` , Capítulo 37) ou adquire um novo recurso reservável.

## 26.3. Chave de Sessão

A chave de assinatura de sessão ( `core/.secret_key` ) é gerada automaticamente na primeira execução do servidor, caso não exista, por meio de `secrets.token_bytes(32)` , e persistida em disco para que sessões permaneçam válidas entre reinicializações do processo. O instalador explicitamente ignora esse arquivo ao copiar o projeto ( `ARQUIVOS_IGNORADOS = {".secret_key"}` ), garantindo que cada instalação gere sua própria chave, nunca reaproveitando uma chave de outra instância ou do repositório de desenvolvimento.

# Sistema de Permissões

## 27.1. Modelo de Perfis

O controle de acesso do sistema é baseado em perfil único por usuário (não há papéis múltiplos combinados nem hierarquia de herança entre perfis). Os seis perfis existentes na versão documentada são: `admin`, `professor`, `portaria`, `bibliotecaria`, `coordenadora` e `secretaria`.

## 27.2. Mapa de Módulos por Perfil

O dicionário `MODULOS_POR_PERFIL`, definido em `core/auth.py`, é a fonte única de verdade sobre quais módulos aparecem no painel inicial (`dashboard.html`) de cada perfil. Este mapa controla apenas a **visibilidade** dos atalhos no painel — a efetiva restrição de acesso a cada rota é imposta, de forma independente, pelos decoradores `@perfil_obrigatorio(...)` aplicados a cada endpoint dentro do respectivo módulo.

**AVISO**

É essencial que toda nova rota sensível seja protegida diretamente por `@perfil_obrigatorio` ou, no mínimo, por `@login_obrigatorio`. A ausência de uma rota no dicionário `MODULOS_POR_PERFIL` apenas oculta o atalho visual — não impede que um usuário autenticado com outro perfil acesse a URL diretamente, caso a rota não tenha, ela mesma, um decorador de controle de acesso.

## 27.3. Perfil Administrador

O perfil `admin` é o único com acesso ao módulo de Administração (gestão de usuários e backups) e é o único perfil cadastrado automaticamente pelo instalador; todos os demais usuários são cadastrados posteriormente pela própria interface web, exclusivamente por um usuário já autenticado como `admin`.

## 27.4. Regra de Unicidade de Senha

Como consequência do modelo de autenticação por senha única (Seção 5.2), o sistema de permissões depende diretamente da unicidade global de senhas entre todos os usuários ativos. Essa regra é verificada tanto na criação (`criar_usuario`) quanto na redefinição (`redefinir_senha`) de credenciais, e não deve ser removida sem uma reformulação simultânea do mecanismo de login para identificação explícita de usuário (por exemplo, reintroduzindo um campo de nome de usuário no formulário de login).

# Procedimentos de Instalação

A instalação do sistema é conduzida pelo script interativo de linha de comando `instalar.py`, na versão 2.0.0 documentada neste manual, e segue as etapas descritas nesta seção.

## 28.1. Pré-requisitos

- Computador com Windows, destinado a permanecer ligado e conectado à rede da instituição durante o período de funcionamento do sistema.
- Python 3.10 ou superior instalado, com `pip` disponível — verificado automaticamente pelo instalador (`verificar_python()`, `verificar_pip()`), que interrompe a instalação caso a versão seja inferior à exigida.
- Prompt de Comando executado com privilégios de Administrador, necessário para a configuração opcional de inicialização automática com o Windows.

## 28.2. Sequência de Instalação

1. Cópia recursiva de todos os arquivos do projeto para a pasta de destino (padrão `C:\SmartCampus`), excluindo pastas técnicas como `__pycache__`, `.git`, `backup` e `logs`, bem como o arquivo `.secret_key`, se presente na origem.
2. Instalação opcional das dependências Python listadas em `DEPENDENCIAS` (`flask`, `bcrypt`, `requests`, `pywhatkit`, `pywin32`), via `pip install`.
3. Coleta da identidade da instituição (nome completo, sigla, bairro/cidade) e gravação em `core/config.json`.
4. Coleta dos parâmetros do servidor (porta de escuta, padrão 80).
5. Cadastro do usuário administrador — o único usuário criado pelo instalador — com senha informada manualmente ou gerada automaticamente por `gerar_senha()`.
6. Cadastro interativo dos recursos reserváveis do módulo Agendamento, gravados em `modulos/agendamento/recursos.json`.
7. Cadastro interativo dos dispositivos IoT (sensores de água, portões, ar-condicionado), com geração de um token único por dispositivo via `gerar_token()` (`secrets.token_hex(12)`), gravados em `modulos/iot/dispositivos.json`.
8. Inicialização das tabelas do SCEDS (`inicializar_banco()`), criando os esquemas de todas as tabelas do sistema.
9. Configuração opcional de inicialização automática do servidor junto com o Windows, via Agendador de Tarefas (`configurar_task_scheduler()`).
10. Exibição e gravação do relatório final de instalação, incluindo a senha do administrador, em `logs/instalacao.txt`.

**AVISO**

A senha do administrador é exibida apenas uma vez, ao final da instalação, e não pode ser recuperada posteriormente — apenas redefinida por um administrador já autenticado, ou, na ausência de qualquer administrador funcional, por intervenção manual direta na tabela `usuarios` do SCEDS. A equipe técnica responsável pela instalação deve orientar a instituição a anotar essa senha imediatamente.



## 28.3. Primeira Execução

Após a instalação, o servidor é iniciado manualmente por `python app.py`, a partir da pasta de destino. O acesso subsequente, de qualquer computador da rede da instituição, é feito via navegador no endereço `http://[IP-do-servidor]`, autenticando-se com a senha do administrador.

# Configuração do Ambiente

## 29.1. Dependências de Execução

| Pacote    | Obrigatoriedade                       | Função                                                                                                     |
|-----------|---------------------------------------|------------------------------------------------------------------------------------------------------------|
| flask     | Obrigatório                           | Framework web de todo o sistema                                                                            |
| bcrypt    | Recomendado (com fallback automático) | Hashing seguro de senhas; na ausência, o sistema utiliza SHA-256 com salt, com nível de segurança inferior |
| requests  | Utilizado por scripts auxiliares      | Chamadas HTTP de apoio (por exemplo, testes manuais dos endpoints IoT)                                     |
| pywhatkit | Opcional                              | Envio de mensagens de WhatsApp pelo módulo Sinal                                                           |
| pywin32   | Específico do Windows                 | Integração com o Agendador de Tarefas do Windows para inicialização automática                             |

## 29.2. Variáveis e Caminhos Sensíveis ao Ambiente

O sistema pressupõe execução em Windows, refletido em convenções de caminho com barra invertida no arquivo de configuração padrão ( "caminho\_base": "C:\\SmartCampus" ) e no uso de pywin32 para inicialização automática. Uma eventual portabilidade para Linux exigiria, no mínimo: revisão de todos os caminhos absolutos codificados em arquivos de configuração; substituição do mecanismo de inicialização automática baseado em Agendador de Tarefas do Windows por um equivalente (por exemplo, um serviço systemd ); e testes de compatibilidade da dependência pywhatkit , que depende de automação de interface gráfica do navegador.

## 29.3. Estrutura de Rede

O servidor deve ser acessível, na porta configurada, por todos os computadores da rede local da instituição que necessitem operar algum módulo. Recomenda-se a atribuição de um endereço IP fixo (ou reserva por DHCP) ao computador servidor, para que o endereço utilizado pelos demais computadores e pelos dispositivos ESP32 não se altere entre reinicializações da rede.

# Processo de Atualização

O sistema, na versão documentada, não dispõe de mecanismo automatizado de atualização (não há verificação de nova versão, download automático ou aplicação de patches). A atualização de uma instalação existente é um procedimento manual, descrito nesta seção como referência até que um mecanismo automatizado seja eventualmente desenvolvido.

## 30.1. Procedimento Recomendado de Atualização

1. Executar um backup completo pelo módulo Administração (Capítulo 31) antes de qualquer alteração de código.
2. Interromper o processo do servidor (`app.py`) e, se em execução, o processo satélite do módulo Sinal (`sinal.pyw`).
3. Substituir os arquivos de código-fonte pela nova versão, preservando integralmente as pastas `sceds/data/`, `logs/`, o arquivo `core/config.json`, o arquivo `core/.secret_key` e os arquivos de configuração específicos de módulo (`recursos.json`, `dispositivos.json`, `turmas.json`).
4. Comparar o novo `core/config.json` de referência (se distribuído) com o arquivo em produção, aplicando manualmente apenas os campos novos, sem sobrescrever os valores específicos da instituição.
5. Caso a nova versão introduza tabelas SCEDS adicionais, executar os comandos `CRIAR TABELA` correspondentes antes de reiniciar o servidor — o motor não cria tabelas automaticamente na ausência do arquivo de esquema.
6. Reiniciar o servidor e verificar `logs/servidor.log` para confirmar que todos os módulos foram registrados com sucesso.

**AVISO**

O SCEDS não possui ferramenta de migração de esquema (equivalente a `ALTER TABLE`). Qualquer alteração de esquema em uma tabela existente (adição de coluna, mudança de tipo) exige a escrita de um script de migração específico, que leia os registros existentes pelo esquema antigo, produza os registros no novo formato e regrave tanto o arquivo de esquema quanto o arquivo de dados. Nenhuma migração desse tipo deve ser executada sem um backup completo prévio.

# Processo de Backup

## 31.1. Backup pela Interface Administrativa

O módulo Administração oferece backup completo sob demanda, acionado pelo endpoint `POST /admin/api/backup/executar`, restrito ao perfil `admin`. A operação invoca `SCEDS.backup_completo()`, que copia o arquivo `.sceds` de cada tabela existente para uma pasta identificada por data e hora (`backup_AAAAMMDD_HHMMSS/`), e em seguida compacta essa pasta em um arquivo `.zip` por meio da biblioteca padrão `zipfile`.

## 31.2. Gerenciamento de Backups Existentes

| Endpoint                                             | Método | Função                                                                                          |
|------------------------------------------------------|--------|-------------------------------------------------------------------------------------------------|
| <code>/admin/api/backup/historico</code>             | GET    | Lista até 30 backups mais recentes, com tabela, tamanho e disponibilidade do arquivo compactado |
| <code>/admin/api/backup/executar</code>              | POST   | Executa um novo backup completo                                                                 |
| <code>/admin/api/backup/download/&lt;nome&gt;</code> | GET    | Envia o arquivo <code>.zip</code> de um backup específico para download                         |
| <code>/admin/api/backup/&lt;nome&gt;</code>          | DELETE | Remove a pasta e o arquivo <code>.zip</code> de um backup específico                            |

## 31.3. Escopo do Backup

O backup completo abrange exclusivamente os arquivos de dados (`.sceds`) das tabelas do SCEDS. Não são incluídos automaticamente: os arquivos de esquema (`.schema.json`), os arquivos de configuração de módulo (`recursos.json`, `dispositivos.json`, `turmas.json`), o arquivo `core/config.json`, nem os logs.

**AVISO**

Como os arquivos de esquema não fazem parte do backup automático, uma restauração completa a partir de um backup exige que os arquivos `.schema.json` correspondentes já existam no destino da restauração (por exemplo, provenientes de uma instalação nova com as tabelas já criadas) ou sejam recompostos manualmente. A equipe técnica deve considerar, como melhoria futura, a inclusão dos arquivos de esquema e dos arquivos de configuração de módulo no procedimento de backup, de modo que um único arquivo compactado seja suficiente para uma restauração completa em um servidor novo.

## 31.4. Periodicidade Recomendada

Não há agendamento automático de backup na versão atual do sistema; a execução depende de acionamento manual pelo administrador pela interface web. Recomenda-se a definição de uma rotina institucional de backup manual diário ou semanal, conforme o volume de uso, até que um agendamento automático seja eventualmente implementado.

# Recuperação de Dados

## 32.1. Restauração a partir de Backup

1. Interromper o processo do servidor.
2. Extrair o arquivo `.zip` do backup desejado (obtido via `/admin/api/backup/download/<nome>` ou diretamente da pasta `backup/` no servidor).
3. Copiar os arquivos `.sceds` extraídos para `sceds/data/`, sobrescrevendo os arquivos atuais das tabelas correspondentes.
4. Confirmar que os arquivos `.schema.json` correspondentes já estão presentes em `sceds/data/` (o backup não os inclui, conforme Seção 31.3); caso a instalação de destino seja nova, os esquemas podem precisar ser recriados via `CRIAR TABELA` antes da cópia dos dados.
5. Reiniciar o servidor e validar, por amostragem, que os registros restaurados aparecem corretamente nos painéis dos módulos afetados.

## 32.2. Recuperação de Registro Individual Corrompido

Como os arquivos `.sceds` são JSON legível, um registro isoladamente corrompido (por exemplo, por edição manual incorreta) pode ser corrigido diretamente em um editor de texto, desde que a estrutura geral do arquivo (chaves `"registros"` e `"proximo_id"`) seja preservada e o arquivo resultante continue sendo um JSON válido. Recomenda-se sempre interromper o servidor antes de editar manualmente um arquivo `.sceds`, para evitar que uma escrita concorrente do próprio sistema sobrescreva a edição manual, ou vice-versa.

**AVISO**

O SCEDS regrava o arquivo de dados inteiro a cada operação de escrita. Uma interrupção abrupta do processo (queda de energia, encerramento forçado) durante a gravação pode, em tese, resultar em um arquivo JSON truncado e inválido. Não há, na versão atual, escrita atômica por arquivo temporário e renomeação (*write-then-rename*), o que constitui um risco residual documentado — mitigado, na prática, pela frequência de backup recomendada na Seção 31.4, mas que deve ser considerado prioritário em uma futura revisão do `SCEDSEngine._salvar()`.

## 32.3. Reconstrução de Índice de Próximo Identificador

Caso o campo `proximo_id` de uma tabela seja perdido ou fique desalinhado em relação aos identificadores já presentes nos registros (por exemplo, após uma edição manual), ele pode ser recalculado manualmente como o maior valor de `id` presente em `"registros"`, somado a um.

# Estratégias de Versionamento

---

## 33.1. Versionamento do Sistema

O sistema identifica sua versão por meio do campo `versao` em `core/config.json` (exposto no endpoint `/api/status`) e, de forma independente, do campo `__version__` do pacote `sceds`. Não há, na versão documentada, garantia formal de que ambos os números avancem em sincronia; recomenda-se que qualquer processo de liberação (release) futuro trate a versão do sistema como número único, atualizado em ambos os locais simultaneamente.

## 33.2. Versionamento de Esquema de Dados

Cada arquivo `.schema.json` registra a data de criação da tabela (`"criada_em"`), mas não um número de versão de esquema. Alterações estruturais em uma tabela existente (Capítulo 30) não deixam, portanto, rastro automático de qual versão do esquema um determinado arquivo de dados segue. Recomenda-se, para evolução futura, a adoção de um campo explícito de versão de esquema em cada arquivo `.schema.json`, incrementado a cada migração estrutural.

## 33.3. Controle de Versão de Código-Fonte

O pacote de instalação distribuído para as instituições não inclui histórico de controle de versão (a pasta `.git` é explicitamente excluída pelo instalador). A árvore de desenvolvimento mantida pela equipe técnica deve, no entanto, ser versionada por um sistema de controle de versão distribuído, com uma tag ou branch de release correspondendo a cada pacote de instalação efetivamente distribuído a uma instituição, de modo a permitir a reprodução exata do código em produção em caso de necessidade de diagnóstico.

# Diretrizes para Desenvolvimento Futuro

---

Esta seção reúne recomendações para equipes que venham a estender o sistema, decorrentes das limitações estruturais identificadas ao longo deste manual.

## 34.1. Consolidação da Função de Acesso a Dados

A função auxiliar `_get_db()`, hoje repetida com pequenas variações em cada módulo (Seção 7.2), deve ser consolidada em um único ponto do núcleo (por exemplo, `core/db.py`), reduzindo duplicação e centralizando qualquer futura mudança na forma como a instância de `SCEDS` é obtida ou configurada.

## 34.2. Escrita Atômica no SCEDS

Conforme discutido na Seção 32.2, a introdução de escrita por arquivo temporário seguida de renomeação atômica em `SCEDSEngine._salvar()` elimina o risco de arquivos truncados em caso de interrupção abrupta do processo, com custo de implementação reduzido frente ao benefício de integridade.

## 34.3. Backup Abrangente

Como recomendado na Seção 31.3, o procedimento de backup deve evoluir para incluir os arquivos de esquema e os arquivos de configuração de módulo, tornando cada backup completo autossuficiente para uma restauração total em um servidor novo.

## 34.4. Agendamento Automático de Backup

A introdução de um agendamento automático de backup (por exemplo, reaproveitando o padrão de thread daemon já utilizado pelo agendador de automações IoT, Seção 20.2) reduziria a dependência de disciplina operacional manual por parte do administrador da instituição.

## 34.5. Extensão Cuidadosa da Linguagem SCEDS

Qualquer extensão da sintaxe textual do SCEDS deve seguir as recomendações da Seção 22.3, preservando compatibilidade retroativa e avaliando, desde o início, se a abordagem por expressão regular continua adequada à complexidade desejada.

## 34.6. Consideração sobre Escala de Concorrência

Caso o número de usuários simultâneos ou o volume de dados por tabela cresça de forma significativa, a equipe responsável deve reavaliar, em conjunto, três decisões arquiteturais interligadas: o servidor WSGI de desenvolvimento (Seção 20.1), o bloqueio por processo do SCEDS (Seção 12.5) e a ausência de índices (Seção 23.2). Essas três limitações foram aceitas conscientemente para o perfil de uso de uma única instituição de porte típico, e não devem ser tratadas isoladamente sem considerar as demais.

# Boas Práticas

---

- **Nunca editar manualmente um arquivo `.sceds` com o servidor em execução.** A gravação concorrente pode resultar em perda da edição manual.
- **Executar um backup completo antes de qualquer alteração estrutural** — atualização de versão, migração de esquema ou edição manual de dados.
- **Preservar a convenção de idioma português** em todo novo código, mensagens e comentários, para manter a coerência do projeto (Seção 8.1).
- **Não introduzir importações cruzadas entre módulos funcionais.** Dados compartilhados entre módulos devem ser obtidos por leitura direta das tabelas SCEDS, nunca por importação do arquivo de API de outro módulo (Seção 9.1).
- **Proteger toda rota nova com o decorador de acesso apropriado** (`@login_obrigatorio` ou `@perfil_obrigatorio`), independentemente de a rota aparecer ou não no menu de módulos do perfil (Seção 27.2).
- **Verificar os logs após toda atualização de código**, à procura de módulos que não tenham sido registrados com sucesso (Seção 24.3).
- **Não confiar na conversão automática de tipos do SCEDS** para validar dados de origem externa (formulários, dispositivos IoT); validar explicitamente no código do módulo (Seção 12.4).
- **Manter os arquivos de configuração de módulo** (`recursos.json`, `dispositivos.json`, `turmas.json`) **sob backup manual**, já que não fazem parte do backup automático do SCEDS.



# Recomendações de Segurança

## 36.1. Autenticação e Senhas

- Preferir sempre a disponibilidade do pacote `bcrypt` no ambiente de execução; o fallback para SHA-256 com salt (Seção 5.2) deve ser tratado como medida de contingência, não como configuração permanente recomendada.
- Orientar a instituição a redefinir a senha do administrador inicial após a primeira instalação, especialmente se a senha foi comunicada verbalmente ou por canal não seguro durante a implantação.
- Preservar a regra de unicidade de senha entre usuários (Seção 27.4) enquanto o modelo de login por senha única estiver em vigor.

## 36.2. Rede e Exposição de Endpoints

- O servidor escuta, por padrão, em todas as interfaces de rede (`host_api: "0.0.0.0"`). A instituição deve garantir que a rede local não seja acessível externamente sem controles adicionais (firewall, VPN), pois o sistema não implementa, por si só, segmentação de rede.
- Os endpoints de ingestão de dados dos dispositivos IoT (`/iot/agua/<sensor_id>`, entre outros) são protegidos por token estático transmitido no cabeçalho `X-Device-Token`, e não por um mecanismo de autenticação renovável (como tokens de curta duração ou assinatura por requisição). A rotação desses tokens, quando necessária, exige atualização manual simultânea em `dispositivos.json` e no firmware de cada ESP32 correspondente.

## 36.3. Dados Sensíveis

- A tabela `usuarios` nunca deve expor o campo `senha_hash` em nenhuma resposta de API; a função `listar_usuarios()` já filtra esse campo por padrão, e qualquer nova rota que liste usuários deve preservar esse filtro.
- O arquivo `logs/instalacao.txt` contém a senha inicial do administrador em texto claro. Esse arquivo deve ser protegido por controle de acesso do sistema operacional e, idealmente, removido ou substituído por uma versão redigida após a senha inicial ser redefinida.
- Tokens de dispositivos IoT (`dispositivos.json`) concedem capacidade de reportar leituras e alterar estados físicos (portões, ar-condicionado); esse arquivo deve receber o mesmo nível de proteção de acesso que os arquivos de configuração de autenticação.

## 36.4. Superfície de Automação Física

**AVISO**

O módulo IoT tem capacidade de acionar portões físicos. Qualquer alteração futura ao protocolo de autenticação de dispositivos, ou à lógica de automação programada (Seção 5.4), deve ser tratada com o mesmo rigor de revisão aplicado a mudanças de controle de acesso do restante do sistema, dado o potencial de impacto físico e de segurança patrimonial de uma falha nesse módulo.

# Solução de Problemas Frequentes

| Sintoma                                                                             | Causa provável                                                                                                                                              | Procedimento                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Um módulo não aparece no menu nem responde às suas rotas                            | Falha de importação do módulo durante o registro de blueprints                                                                                              | Consultar <code>logs/servidor.log</code> à procura de mensagens "não pôde ser importado" ou "Erro ao registrar" (Seção 24.3)                                                                                                            |
| Login falha mesmo com a senha correta                                               | Usuário desativado ( <code>ativo: false</code> ), ou senha duplicada gerando ambiguidade no momento da criação                                              | Verificar o registro do usuário na tabela <code>usuarios</code> via Administração > Usuários                                                                                                                                            |
| Alunos aparecem com turma, série ou curso em branco no módulo Ocorrências           | Registro criado antes da padronização atual da lista de turmas, ou inserido sem esses campos                                                                | Executar <code>python modulos/ocorrencias/reparar_alunos.py</code> , que completa automaticamente os campos ausentes a partir de <code>turmas.json</code> quando possível, e marca com "(a definir)" os casos que exigem revisão manual |
| Ocorrências antigas aparecem com tipo não reconhecido pela interface                | Registros gravados antes da padronização dos identificadores de tipo (Seção 18)                                                                             | Executar <code>python modulos/ocorrencias/normalizar_tipos.py</code> , que aplica <code>normalizar_tipo()</code> a todas as ocorrências existentes                                                                                      |
| Alunos com turma fora do padrão vigente após reorganização de turmas da instituição | A instituição renomeou ou reorganizou turmas sem migrar os cadastros existentes                                                                             | Executar <code>python modulos/ocorrencias/verificar_turmas.py</code> para identificar os alunos afetados e seguir as duas opções de correção que o script apresenta                                                                     |
| Dispositivo ESP32 não consegue reportar leituras ou receber comandos                | Token incorreto no firmware, ou identificador de dispositivo ( <code>sensor_id</code> , <code>pid</code> ) não cadastrado em <code>dispositivos.json</code> | Confirmar correspondência exata entre o token configurado no firmware <code>.ino</code> e o token cadastrado; confirmar que o identificador usado na URL corresponde a um dispositivo listado                                           |
| Sinal sonoro não toca no horário programado                                         | Processo <code>sinal.pyw</code> não está em execução, ou lista de horários incorreta                                                                        | Verificar <code>logs/sinal.log</code> ; confirmar que o processo satélite está ativo e acessível pelo proxy em <code>api_sinal.py</code>                                                                                                |
| Backup não pode ser restaurado em servidor novo                                     | Ausência dos arquivos de esquema, não incluídos no backup automático (Seção 31.3)                                                                           | Recriar os esquemas via <code>CRIAR TABELA</code> , ou copiar os arquivos <code>.schema.json</code> de uma instalação de referência, antes de copiar os dados restaurados                                                               |

# Glossário Técnico

| Termo                | Definição                                                                                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Blueprint            | Mecanismo do Flask para organizar rotas em módulos independentes, registrados posteriormente na aplicação principal com um prefixo de URL próprio                         |
| ESP32                | Microcontrolador de baixo custo com conectividade Wi-Fi integrada, utilizado no módulo IoT para leitura de sensores e acionamento de atuadores físicos                    |
| Hash de senha        | Representação irreversível de uma senha, utilizada para verificação de credenciais sem armazenar o valor original em texto claro                                          |
| Perfil               | Categoria funcional atribuída a um usuário (admin, professor, portaria, bibliotecária, coordenadora, secretaria), que determina os módulos visíveis e as rotas acessíveis |
| Polling              | Padrão de comunicação em que um dispositivo consulta periodicamente o servidor para verificar se há uma ação pendente, em vez de o servidor iniciar a comunicação         |
| SCEDS                | Camada de persistência de dados própria do Smart Campus, composta por um motor de armazenamento em arquivos JSON e uma linguagem textual de comandos em português         |
| Schema (esquema)     | Definição estrutural de uma tabela do SCEDS: nome, colunas, tipos e modificadores, armazenada em arquivo <code>.schema.json</code>                                        |
| Token de dispositivo | Cadeia de caracteres única, gerada por dispositivo IoT, utilizada como credencial simples para autenticar as requisições desse dispositivo ao servidor                    |
| WSGI                 | Interface padrão entre servidores web e aplicações Python; o Flask a implementa nativamente por meio de seu servidor de desenvolvimento embutido                          |

# Apêndices

## Apêndice A — Lista Completa de Módulos e Prefixos de Rota

| Módulo              | Arquivo de blueprint                                    | Prefixo                                                                                   |
|---------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Sinal               | <code>modulos/sinal/api_sinal.py</code>                 | <code>/sinal</code>                                                                       |
| Agendamento         | <code>modulos/agendamento/api_agendamento.py</code>     | <code>/agendamento</code>                                                                 |
| Biblioteca          | <code>modulos/biblioteca/api_biblioteca.py</code>       | <code>/biblioteca</code>                                                                  |
| Secretaria/Portaria | <code>modulos/secretaria_portaria/api_sp.py</code>      | <code>/</code> (raiz, com sub-rotas <code>/secretaria/</code> e <code>/portaria/</code> ) |
| IoT                 | <code>modulos/iot/api_iot.py</code>                     | <code>/iot</code>                                                                         |
| Ocorrências         | <code>modulos/ocorrencias/api_ocorrencias.py</code>     | <code>/ocorrencias</code>                                                                 |
| Monitoramento       | <code>modulos/monitoramento/api_monitoramento.py</code> | <code>/monitoramento</code>                                                               |
| Administração       | <code>admin/api_admin.py</code>                         | <code>/admin</code>                                                                       |

## Apêndice B — Scripts de Manutenção Disponíveis

| Script                                 | Localização                       | Finalidade                                                                                                      |
|----------------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>reparar_alunos.py</code>         | <code>modulos/ocorrencias/</code> | Completa automaticamente campos de série/turma/curso ausentes no cadastro de alunos                             |
| <code>normalizar_tipos.py</code>       | <code>modulos/ocorrencias/</code> | Corrige registros antigos de ocorrência cujo campo de tipo não está no formato canônico atual                   |
| <code>verificar_turmas.py</code>       | <code>modulos/ocorrencias/</code> | Identifica alunos cadastrados com turma fora do padrão vigente após reorganização institucional                 |
| <code>simulador_iot_completo.py</code> | <code>modulos/iot/</code>         | Simula o comportamento dos dispositivos ESP32 (sensores e atuadores) para testes sem hardware físico disponível |
| <code>teste_sceds.py</code>            | <code>sceds/</code>               | Casos de teste manuais do motor e do interpretador SCEDS                                                        |

## Apêndice C — Modelo de Ficha de Comando SCEDS (para novos comandos)

Comando: \_\_\_\_\_

Descrição técnica: \_\_\_\_\_

Sintaxe completa: \_\_\_\_\_

Parâmetros: \_\_\_\_\_

Valor de retorno: \_\_\_\_\_

Limitações conhecidas: \_\_\_\_\_

Exemplo comentado: \_\_\_\_\_

# Referências Técnicas

## 40.1. Documentação Externa de Base

- Documentação oficial do framework Flask, para aprofundamento em blueprints, contexto de aplicação, sessões e manipuladores de erro.
- Documentação oficial da biblioteca `bcrypt` para Python, para detalhes do algoritmo de hashing utilizado na autenticação.
- Documentação de referência da linguagem Python 3 (versão 3.10 ou superior), para as construções de tipagem ( `str` | `None` , `list[dict]` ) empregadas em todo o código do sistema.
- Documentação do ambiente Arduino/ESP32, para manutenção e extensão do firmware dos dispositivos físicos do módulo IoT.

## 40.2. Documentos Internos Relacionados

| Documento                                | Localização                                                 | Conteúdo                                                                                                                                             |
|------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Planejamento de Hardware IoT             | <code>modulos/iot/hardware/PLANEJAMENTO_HARDWARE.md</code>  | Lista de compras, esquemas de ligação elétrica e especificações de montagem dos dispositivos ESP32                                                   |
| Instruções de Instalação (usuário final) | <code>LEIA-ME.md</code> , na raiz do pacote de distribuição | Roteiro resumido de instalação, destinado à equipe que implanta o sistema em uma nova instituição, sem pressupor conhecimento técnico de programação |

## 40.3. Manutenção deste Manual

Este manual deve ser revisado a cada alteração estrutural do sistema — adição ou remoção de módulo, alteração de esquema de dados, extensão da linguagem SCEDS, ou mudança de procedimento de instalação, atualização ou backup. A revisão deve preservar a numeração de capítulos sempre que possível, adicionando subseções em vez de renumerar capítulos existentes, para manter a estabilidade de referências cruzadas em uso pela equipe técnica.